

Chapter 5: System Implementation, Testing and Evaluation

5.1 Introduction

This chapter reports how the design of Chapter 4 was implemented as the CAMR memory engine and its evaluation harness, how the implementation was tested, and what the experiments measured. The project follows the AI and machine learning route of the departmental guidelines with one important difference: the language models were never trained or fine-tuned. Every model ran with frozen weights, and what changed between conditions was the external memory and the engine's policy for using it. The chapter therefore reports dataset provenance, the implemented data-to-answer pipeline, the experimental configuration, the tests that verify the requirements of section 4.2, and the measured results for each research objective.

Every number in this chapter was produced by the code in the project repository on real public benchmark data and real models, and each table names the results file it comes from. Where a result is weaker than expected, or a requirement was not met, it is reported as measured. Section 5.2 describes the environment, section 5.3 the datasets, section 5.4 the implementation, section 5.5 testing, section 5.6 the version-control evidence, section 5.7 the declaration of AI-assisted tools, and section 5.8 the results, discussion and limitations.

The work proceeded in increments, each ending with a measured result that decided the next step. The proposal's engine and harness were built and unit-tested first. The engine was then made capability-adaptive (gating, entity bridging and procedural memory), and a residual-gap condition was added. Next came a real-model pilot on a CPU-only machine, a memory-growth experiment, a model-size sweep spread across five machines, and a comparison with a frontier cloud model (Kimi K3). The engine's per-question policy was then learned from rewards, and the multi-hop result was confirmed on 200 held-out questions. Finally, the engine was packaged as a downloadable desktop application. Figure 5.1 shows this sequence as it appears in the repository's commit history, and Figure 5.2 shows the experimental pipeline that every result in this chapter passed through.

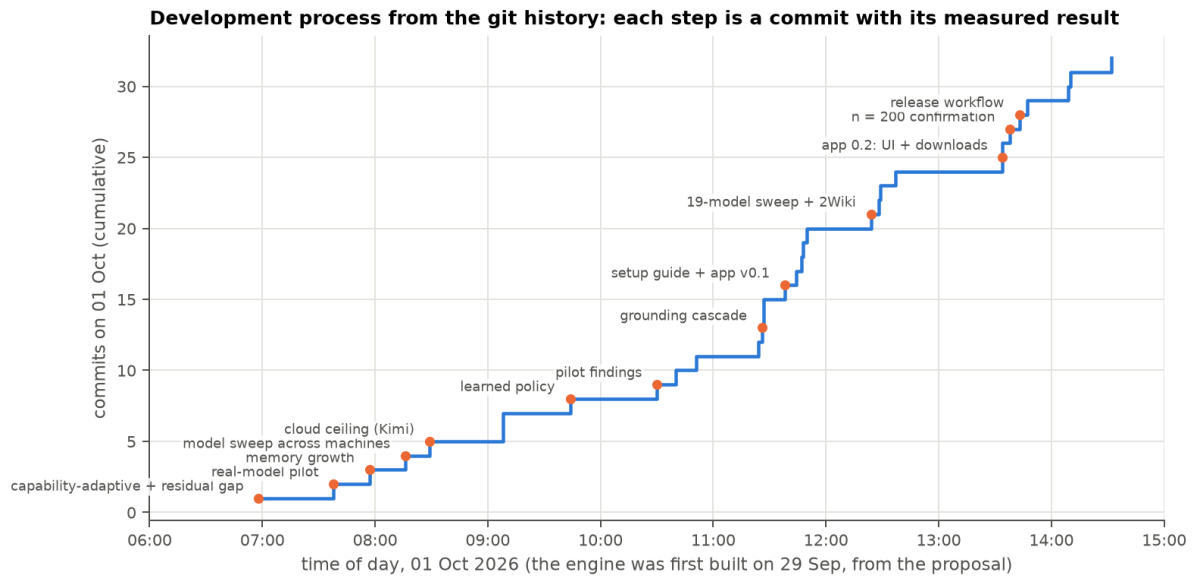


Figure 5.1: Development process reconstructed from the git history; each marked step is a commit that added an implemented or measured increment

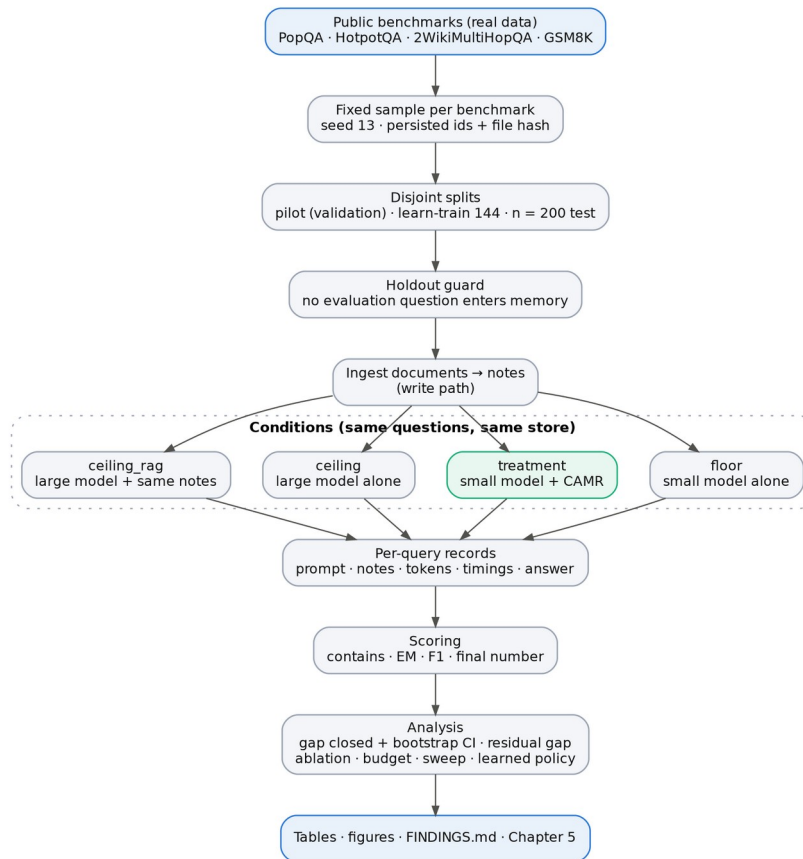


Figure 5.2: Experimental pipeline: from public benchmark data to the tables and figures of this chapter

5.2 Implementation Environment

All development and every experiment ran on Linux machines with x86-64 CPUs and no GPU. This is the consumer-laptop class of device that NFR-01 targets: four CPU cores and 16 GB of memory. The system is a local research prototype. It was not deployed for public access.

5.2.1 Hardware and Deployment Environment

Table 5.1: Implementation environment and minimum requirements

Item or layer	Tested specification or version	Purpose and justification	Evidence
Primary machine (A)	Intel(R) Xeon(R) Processor @ 2.10GHz, 4 logical CPUs, 15.7 GB RAM (14.3 GB usable by processes), no GPU, Linux x86-64	Development, all pilot, growth, ablation, learning and Kimi runs; the stated minimum (NFR-01)	results/pilot/tables/profile.json
Sweep machines (B–E)	Same container class: 4 vCPU Intel Xeon (2.10 GHz; machine C 2.80 GHz), 16 GB RAM	Model-size sweep run in parallel, one model family per machine, then merged	camr merge; FINDINGS F10
Minimum to run the engine	4 cores, 16 GB RAM. Measured peak: engine process 712 MB, model runtime 1987 MB (qwen2.5:0.5b), store 47.8 MB	A 0.5B model and the engine fit in under 3 GB, leaving headroom on a 16 GB device	camr profile (TC-14)
Large local model	gpt-oss:20b (21B total, 3.6B active, MoE) with memory-mapped weights	Local large-model ceiling; loads in 16 GB only with mmap (first attempt was killed at 13.3 GB)	FINDINGS F6
Cloud ceiling	Moonshot Kimi K3 and Kimi K2.6 via an OpenAI-compatible HTTPS API	Frontier reference model (ceiling condition only; NFR-06)	configs/pilot_kimi.yaml
End-user device (CAMR Personal)	Windows 10/11 (x64, ARM64), macOS (Apple Silicon, Intel), Linux (x64, ARM64); Ollama installed; ≥ 4 GB RAM for a 1.5B model	The application tested by real users; talks only to Ollama on the same machine	CI build and launch test on all six (Table 5.11)
Specialised hardware	None: no GPU, NPU or accelerator was used anywhere	The study targets CPU-only consumer devices; every timing is a CPU timing	Profiles and CI runner labels

5.2.2 Software, Services and Configuration

Table 5.2: Software and services used

Component	Version	Role in the system
Python	3.11.15	Implementation language of the engine, harness and CLI
Ollama	0.35.0	Local model runtime on the loopback interface (IR-01);

Component	Version	Role in the system
		Q4/Q8 GGUF weights
SQLite / sqlite-vec	3.45.1 / 0.1.9	Single-file store with exact kNN vector search (FR-05, IR-04, NFR-03)
sentence-transformers / PyTorch	6.1.0 / 2.14.1 (CPU)	BGE-small-en-v1.5 embedder, 384 dimensions (IR-03)
NumPy	2.4.6	Scoring, bootstrap confidence intervals, ridge regression and LinUCB
Streamlit	1.64.0	Read-only inspection interface (FR-17, IR-06)
Matplotlib / Playwright	3.11.2 / 1.63.0	Figures; automated screenshots of the inspector
pytest	9.1.1	Unit and integration tests (section 5.5)
Moonshot API	kimi-k3, kimi-k2.6	Cloud ceiling. The key is read from the MOONSHOT_API_KEY environment variable and never written to configuration, logs or the repository

Configuration is a single YAML file per experiment (NFR-07). It is loaded into typed dataclasses that reject unknown keys, so a misspelt ablation parameter fails loudly instead of silently running the default. Each run records the hash of its full configuration. A completed run with the same label and hash is skipped, which makes every command resumable after an interruption. Containers (Docker) were not part of the solution: the experiments ran directly on the hosts above, and the application ships as a single self-contained executable per platform.

5.2.3 Deployment and Reproducibility

The engine installs as a Python package and is driven by one command-line program, `camr` (IR-05). Every result in this chapter can be regenerated with the commands below. They were run as shown, on Linux with Python 3.11 and Ollama 0.35. Each block can be copied into a terminal. Steps 4 and 6 need the cloud key only for the Kimi comparisons; everything else runs offline once models and data are downloaded.

Step 1: get the code and install it (a virtual environment keeps it separate from other Python projects).

```
git clone https://github.com/Satasdin/CAMR.git
cd CAMR
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\
Scripts\activate
pip install -e ".[vec,embed,dev,inspect]"
python -m pytest -q                                    # health check: all
tests pass, no network needed
```

Step 2: install the local models with Ollama (<https://ollama.com/download>).

```
ollama pull qwen2.5:0.5b      # the small model of the pilot
```

```
ollama pull gpt-oss:20b          # large local comparison model (needs ~14 GB RAM)
ollama pull qwen2.5:1.5b && ollama pull llama3.2:3b    # used in the n = 200 study
```

Step 3: download the real benchmark data (PopQA, HotpotQA, 2WikiMultiHopQA, GSM8K) and the Wikipedia corpus for PopQA. The pilot corpus is already in the repository; the learning split needs its own.

```
python scripts/fetch_data.py          # prints a hash per file
python scripts/build_popqa_corpus.py --config configs/learn.yaml --split train --run-dir results/learn \
    --out data/popqa/corpus_learn.jsonl
```

Step 4: the cloud key (only for the Kimi conditions). It is read from an environment variable and never written to any file.

```
export MOONSHOT_API_KEY=...          # your own key
```

Step 5: run the experiments. Each command is resumable: run it again after an interruption and it skips finished work.

```
camr reproduce      --config configs/pilot.yaml          # pilot: ingest, conditions, ablation, budgets, profile
camr grow           --config configs/pilot.yaml --stages 0.25,0.5,1.0
# memory growth
camr sweep-models   --config configs/pilot.yaml --models qwen2.5:0.5b, llama3.2:3b, phi4-mini:3.8b # any list
camr retrieval-eval --config configs/retrieval_eval.yaml # 500 + 500 questions, no LLM
camr learn          --config configs/learn.yaml --phase all
# learned policy
camr ingest         --config configs/hotpot200.yaml
# n = 200 study
camr sweep-models   --config configs/hotpot200.yaml --models qwen2.5:0.5b, qwen2.5:1.5b, llama3.2:3b
```

Step 6: the cloud comparisons.

```
camr run --config configs/pilot_kimi.yaml --out results/pilot --condition ceiling --benchmark popqa --label kimi
camr run --config configs/pilot_kimi.yaml --out results/pilot --condition ceiling_rag --benchmark popqa --label kimi
camr run --config configs/hotpot200.yaml --condition ceiling --benchmark hotpotqa --label kimi
camr run --config configs/hotpot200.yaml --condition ceiling_rag --benchmark hotpotqa --label kimi
```

Step 7: regenerate the tables, figures and this chapter from the logged results alone (NFR-04).

```

camr analyse --run-dir results/pilot
python scripts/make_figures.py && python scripts/make_report_figures.py &&
python scripts/make_diagrams.py
camr inspect --run-dir results/pilot # read-only web view
at http://localhost:8501
python scripts/build_chapter5.py --template proposal.docx --out
build/Chapter5

```

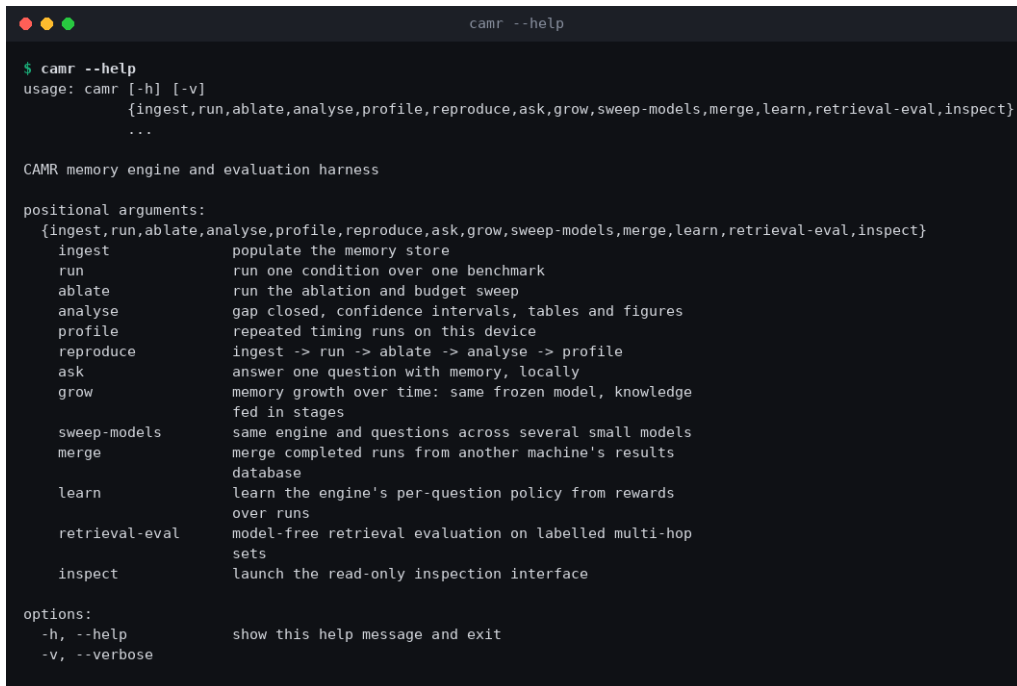
The application only (no experiments): download the file for the platform from the v0.2.1 release page, or install it with Python:

```

pip install "camr[app] @ git+https://github.com/Satasdin/CAMR.git"
ollama pull qwen2.5:1.5b
camr app # opens
http://127.0.0.1:8502

```

Figure 5.3 shows the CLI's command surface and Figure 5.4 the test-suite health check, both captured from real runs.



```

camr --help

$ camr --help
usage: camr [-h] [-v]
           {ingest,run,ablate,analyse,profile,reproduce,ask,grow,sweep-models,merge,learn,retrieval-eval,inspect}
           ...

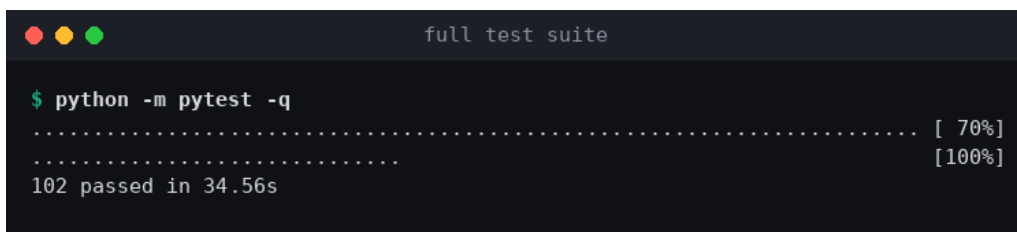
CAMR memory engine and evaluation harness

positional arguments:
  {ingest,run,ablate,analyse,profile,reproduce,ask,grow,sweep-models,merge,learn,retrieval-eval,inspect}
  ingest                populate the memory store
  run                   run one condition over one benchmark
  ablate                run the ablation and budget sweep
  analyse               gap closed, confidence intervals, tables and figures
  profile               repeated timing runs on this device
  reproduce             ingest -> run -> ablate -> analyse -> profile
  ask                   answer one question with memory, locally
  grow                 memory growth over time: same frozen model, knowledge
                       fed in stages
  sweep-models          same engine and questions across several small models
  merge                 merge completed runs from another machine's results
                       database
  learn                 learn the engine's per-question policy from rewards
                       over runs
  retrieval-eval        model-free retrieval evaluation on labelled multi-hop
                       sets
  inspect               launch the read-only inspection interface

options:
  -h, --help            show this help message and exit
  -v, --verbose

```

Figure 5.3: The camr command-line interface (captured output)



```

full test suite

$ python -m pytest -q
..... [ 70%]
..... [100%]
102 passed in 34.56s

```

Figure 5.4: Health check: the offline test suite passing (captured output)

5.3 Dataset Description

Four public benchmarks were used (DR-01). They were downloaded from their official distributions by `scripts/fetch_data.py`, and no data was generated synthetically. Table 5.3 summarises them and Table 5.4 shows one real record of each. PopQA ships without documents, so a corpus of real Wikipedia summaries was fetched for the subject entity of each sampled question (design note A4). HotpotQA and 2WikiMultiHopQA supply gold and distractor paragraphs, and those paragraphs, never the question–answer records, populate memory (DR-03).

Table 5.3: Benchmarks used

Benchmark	Source and licence	Size of the file used	Task type	Metric
PopQA	Mallen et al. (2023); templates over Wikidata facts	14,267 questions; corpus of 357 Wikipedia summaries (CC BY-SA)	Single-hop, long-tail facts	Answer contained (accuracy)
HotpotQA (dev distractor)	Yang et al. (2018); crowd-written over Wikipedia; CC BY-SA	7,405 questions (5,918 bridge, 1,487 comparison), 10 paragraphs each	Multi-hop	Exact match, F1
2WikiMultiHopQA (dev)	Ho et al. (2020); Wikipedia and Wikidata	12,576 questions	Multi-hop (retrieval study only)	Supporting-paragraph recall
GSM8K (test; train for exemplars)	Cobbe et al. (2021); human-written; MIT	1,319 test; 7,473 train solutions	Reasoning control	Final-answer accuracy

Table 5.4: One record from each benchmark (verbatim)

Benchmark	Question	Reference answer	Supporting evidence
PopQA	What is George Rankin's occupation?	politician (also: political leader, political figure)	Wikipedia summary of <i>George Rankin</i>
HotpotQA	Were Scott Derrickson and Ed Wood of the same nationality?	yes	Paragraphs <i>Scott Derrickson</i> and <i>Ed Wood</i> (gold) plus 8 distractors
GSM8K	Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder ... for \$2 per fresh duck egg. How much in dollars does she make every day?	18	None (reasoning); solved train problems serve as exemplars

5.3.1 Data Preparation and Partitioning

Each benchmark sample was drawn once from seed 13, sorted by question identifier before sampling so that the draw does not depend on file order, and persisted to

samples/<benchmark>.json with the source file's hash (DR-02). Every condition and ablation reused the same questions, so all comparisons are paired. The pilot evaluation sample was 30 PopQA, 30 HotpotQA and 12 GSM8K questions (72 in total), limited by the time the 20B CPU ceiling needed (up to 150 s per GSM8K answer). For the learned policy, a **disjoint** training split of 144 questions (60 PopQA, 60 HotpotQA, 24 GSM8K) was drawn from the questions not in the evaluation sample. The code asserts that the two splits do not overlap.

Memory was built from documents only. The HotpotQA store also took the paragraphs of 100 further, unsampled questions as extra distractors (design note A5). Before ingestion a holdout guard scanned every document and refused any that contained an evaluation question verbatim (DR-04, TC-21). Text was normalised (Unicode NFKC, whitespace) and paragraphs were stored verbatim with their title prefixed, e.g. *The Loudwater Mystery (film): The Loudwater Mystery is a 1921 British silent...* The structured (distilled) write policy is implemented and tested, but it was not used for the reported runs, because distilling every paragraph with the 0.5B model exceeded the pilot's time budget.

Table 5.5 follows one real record from the raw files to what the model finally reads.

Table 5.5: One PopQA record before and after preparation

Stage	Content
Raw question (test.tsv)	id 5939255 · subject <i>The Loudwater Mystery</i> · property <i>director</i> · question <i>Who was the director of The Loudwater Mystery?</i> · answers ["Walter West", "Walter Alabaster West"]
Raw document (corpus)	{"title": "The Loudwater Mystery (film)", "text": "The Loudwater Mystery is a 1921 British silent crime film directed by Walter West and starring Gregory Scott, Pauline Peters and Clive Brook. It was based on the 1920 novel ..."}
Stored note (#4863)	<i>The Loudwater Mystery (film): The Loudwater Mystery is a 1921 British silent crime film directed by Walter West ...</i> (47 tokens; 384-d BGE vector; provenance: dataset popqa, document id, ingestion time)
Read at question time	Similarity 0.77 → admitted; next candidate 0.62 → outside the 0.15 margin. The prompt's notes section contains this single note (47 tokens instead of a 512-token budget).
Answer and score	0.5B alone: <i>John Sturges</i> (wrong). 0.5B + CAMR: <i>Walter West</i> : contains = 1, EM = 1

5.4 System Implementation

Figure 5.5 shows the implemented architecture. Everything inside the engine runs on the device. The cloud model is used only as a measurement ceiling and as the escalation target of the learned policy.

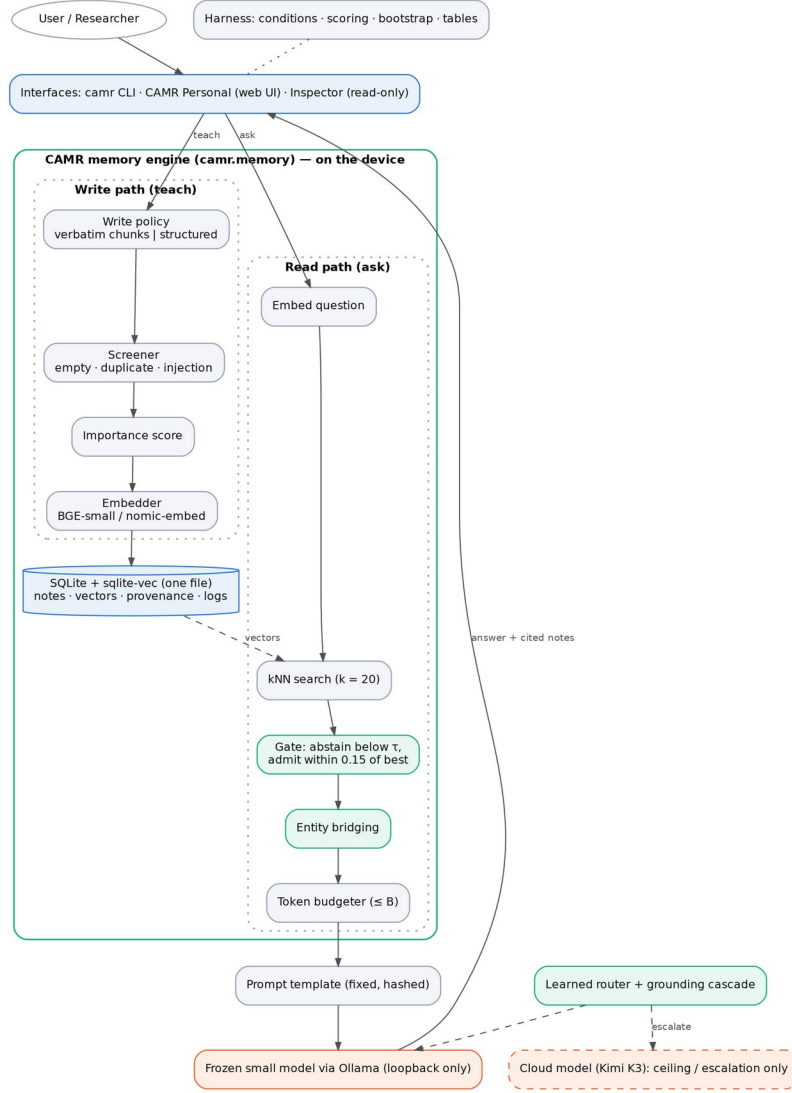


Figure 5.5: Implemented CAMR architecture: write path, single-file store, capability-adaptive read path (green), frozen local model and optional cloud escalation

The implementation follows the package structure of Figure 4.6: `camr.memory` (the engine), `camr.models` (runners and prompt templates), `camr.harness` and `camr.eval` (experiments and analysis), `camr.learn` (the learned policy), `camr.cli` and `camr.inspect`. A test enforces the dependency rules of the package diagram (memory imports nothing from harness, eval, cli or inspect, and nothing imports inspect).

5.4.1 Memory Engine: Write Path

Ingestion implements the sequence of Figure 4.4. A write policy turns a source document into candidate notes (FR-01). The screener rejects empty, degenerate, duplicate, over-length and injection-like candidates and logs the reason (FR-02). An importance score is assigned (FR-04). Then the note, its embedding and its provenance are committed in one SQLite

transaction (FR-03, FR-05), so the store never holds a note without its vector or its source. Duplicate detection uses a checksum with a UNIQUE constraint, namespaced by note population, so ablations that change what is stored (write policy, screening) coexist in one file (design note A6). Ingesting 14,567 HotpotQA paragraphs into a 24,475-note store took 622 s on machine A, of which 569 s was embedding.

5.4.2 Memory Engine: Read Path

Recall implements Figure 4.5 and Equations 4.1–4.5. The question is embedded and the $k = 20$ nearest notes are found by exact kNN in sqlite-vec (FR-06). The notes are ranked by the configured policy, similarity only or composite (FR-07), and packed greedily under the token budget (FR-08). Three capability-adaptive mechanisms were added during implementation, each switchable in configuration and each justified by measurement in section 5.8:

- i. **Gating.** The engine abstains when the best note's similarity is below 0.50, and admits only notes within 0.15 of the best. This keeps irrelevant text away from a small reader.
- ii. **Entity-bridge expansion.** When an admitted note mentions the title of another stored entity, that entity's notes are pulled in. This is a one-step associative hop that needs no graph construction and no model call, and it replaces the optional graph layer of FR-18.
- iii. **Procedural memory.** For reasoning questions, solved GSM8K *train* problems are retrieved as worked examples instead of factual notes.

Recency uses a logical clock, which advances one step per question instead of wall-clock time, and access state is reset before every run. This makes recall deterministic and independent of machine speed (NFR-05, design note A1).

5.4.3 Model Runners

All runners expose a single `generate` operation, and none can modify model weights (FR-09). `OllamaRunner` calls the local runtime with greedy decoding (temperature 0, seed 13) and refuses any host but loopback, unless it is explicitly configured as a ceiling (IR-01, NFR-06). `OpenAICompatRunner` calls the Moonshot API with a pinned model identifier, a timeout and bounded exponential-backoff retries. When retries are exhausted the query is logged as failed rather than scored (IR-02, IR-07). The Kimi API accepts only temperature 1, so the cloud ceiling is a single stochastic sample per question; this limitation is reported in section 5.8.

5.4.4 Evaluation Harness and Learned Policy

The harness runs four conditions over the fixed sample: **floor** (small model alone), **treatment** (small model + CAMR), **ceiling** (large model alone) and **ceiling_rag** (large model given the same notes). Each query writes a structured record (FR-13): condition, model and version, configuration hash, retrieved note identifiers and scores, prompt and generated tokens, retrieval, prefill and decode times, and the score. Analysis computes gap closed (Equation 3.1) with paired bootstrap confidence intervals over 2,000 resamples (FR-15). Next to it, it reports the absolute gain, and the **residual gap** (ceiling_rag – treatment): what model size still buys when both models read the same evidence.

The learned policy (camr learn) chooses, per question, one of five actions: answer with the small model alone, with 128, 512 or 1,024 tokens of memory, or escalate to Kimi K3. Its training configuration is given in Table 5.6. No neural network was trained and no model weights changed. The learned parameters are a ridge-regression weight vector per action (5×11 numbers) and a grounding classifier.

Table 5.6: Learned-policy training configuration (from camr/learn/bandit.py and rl_report.json)

Setting	Value
Language-model training	None. All model weights frozen: 0 fine-tuning epochs, no learning rate, no gradient updates
Training / evaluation questions	144 / 72 (disjoint; 60/60/24 and 30/30/12 PopQA/HotpotQA/GSM8K)
Actions (arms)	local; mem128; mem512 (+ bridging); mem1024 (+ bridging); cloud (Kimi K3)
Logged outcomes	Every action on every question: 720 training and 360 evaluation answers
Context features (11)	Bias; task type (3 one-hot); best-note similarity; margin to second note; number of near-best notes; question length; task type $\times$ best-note similarity (3)
Reward	correct – $0.3 \times [\text{cloud call}] - 0.1 \times \text{prompt tokens} / 1,000$
Offline learner	Ridge regression per action, $\lambda = 1.0$, closed-form solution (one pass)
Online learner (LinUCB)	Bandit feedback only; exploration $\alpha = 0.5$; 5 epochs (passes) over the 144 training questions = 720 episodes, shuffled with seed 13; held-out evaluation every 20 episodes (37 checkpoints)
Grounding cascade	Ridge classifier on answer signals (answer words found in the notes, abstention-like text, length); escalation thresholds swept over 0–1.01
Cloud-cost sweep	0, 0.05, 0.1, 0.2, 0.3, 0.5, 0.8, 1.2 (Pareto table)

Figure 5.6 shows the learned decision for one question.

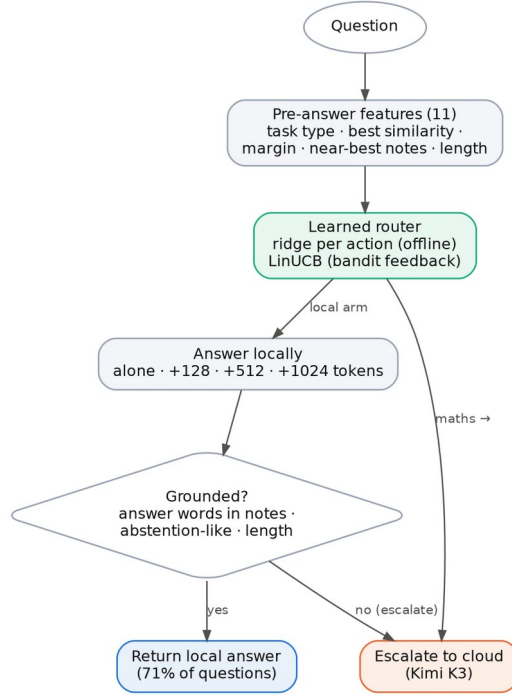


Figure 5.6: Learned engine policy: a router chooses local memory budgets or the cloud from pre-answer features, and a grounding check escalates local answers that are not supported by the notes they read

5.4.5 Model Selection and the Validation–Test Protocol

Because no weights were trained, *selection* meant choosing which frozen model and which engine configuration to recommend, and that choice was made on evidence kept apart from the final test. The **baseline** was each small model answering alone (the floor condition). The **candidates** were 19 quantised models from 0.5B to 14.8B parameters across ten families (Table 5.17), each run with the same store, questions and prompt templates (seed 13, greedy decoding, temperature 0). The pilot sample (30/30/12 questions) and the 500-question retrieval study served as the **validation** evidence: they fixed the read path (similarity ranking, gating at 0.50 with a 0.15 margin, entity bridging) and the budgets (128 tokens for single facts, 512 for multi-hop). The 200-question HotpotQA study (reported in section 5.8) then served as a **held-out test**: its configuration was written to `configs/hotpot200.yaml` before it ran and was not changed afterwards. Every run's configuration, hash, model digest and per-question records are saved in its results database and in `records/*.jsonl`, so each reported number can be regenerated.

5.4.6 Inspection Interface and Integration Workflow

The inspector implements the four wireframes of Figures 4.10–4.13 and opens the results database read-only (mode=ro), so it cannot write (FR-17, IR-06). The screenshots below were captured automatically by Playwright from the real results database. Figure 5.7 is the Run Dashboard of the final pilot. Figure 5.8 follows one question end to end: the 0.5B model alone answered "John Sturges" (wrong), and with CAMR it read one admitted note of 47 tokens and answered "Walter West" (correct). Gating stopped the next candidate (similarity 0.62 vs 0.77) at the budget cutoff.

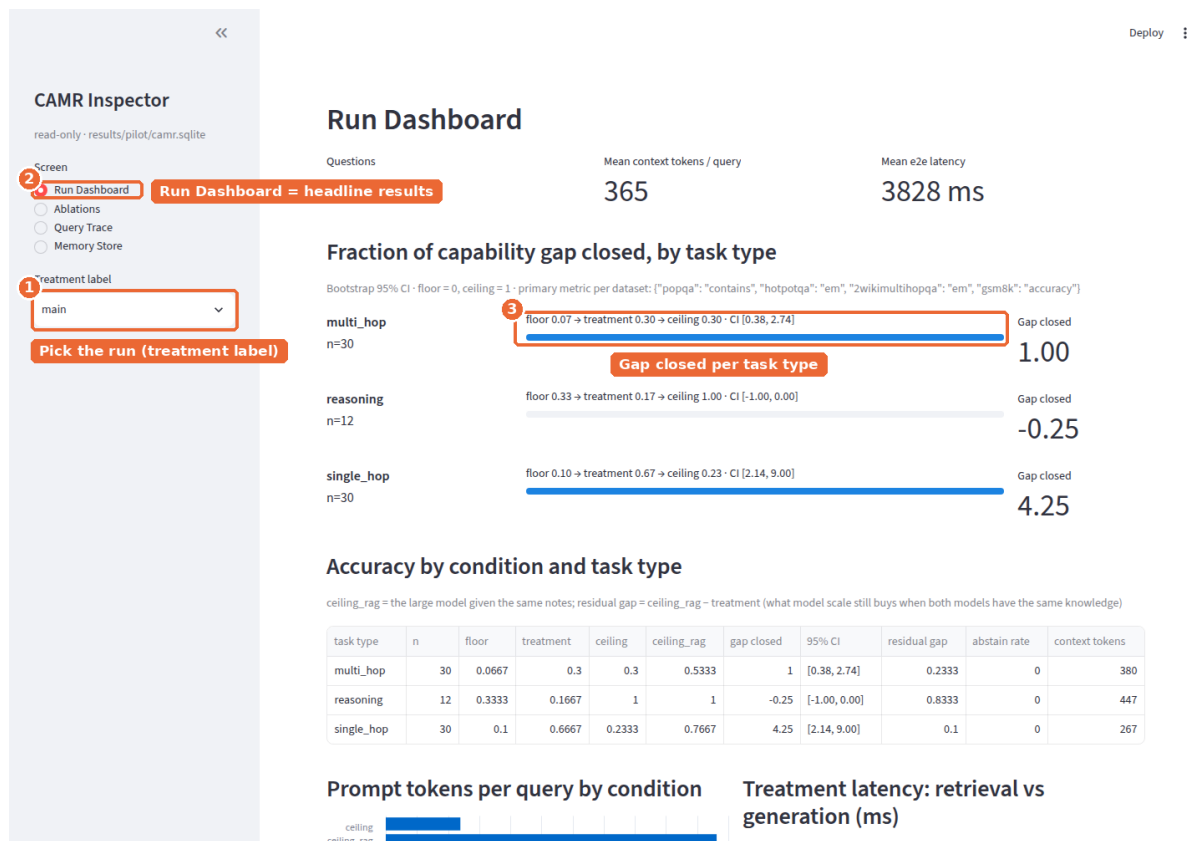


Figure 5.7: Run Dashboard of the final pilot (markers: run selector, screen, gap closed per task type)

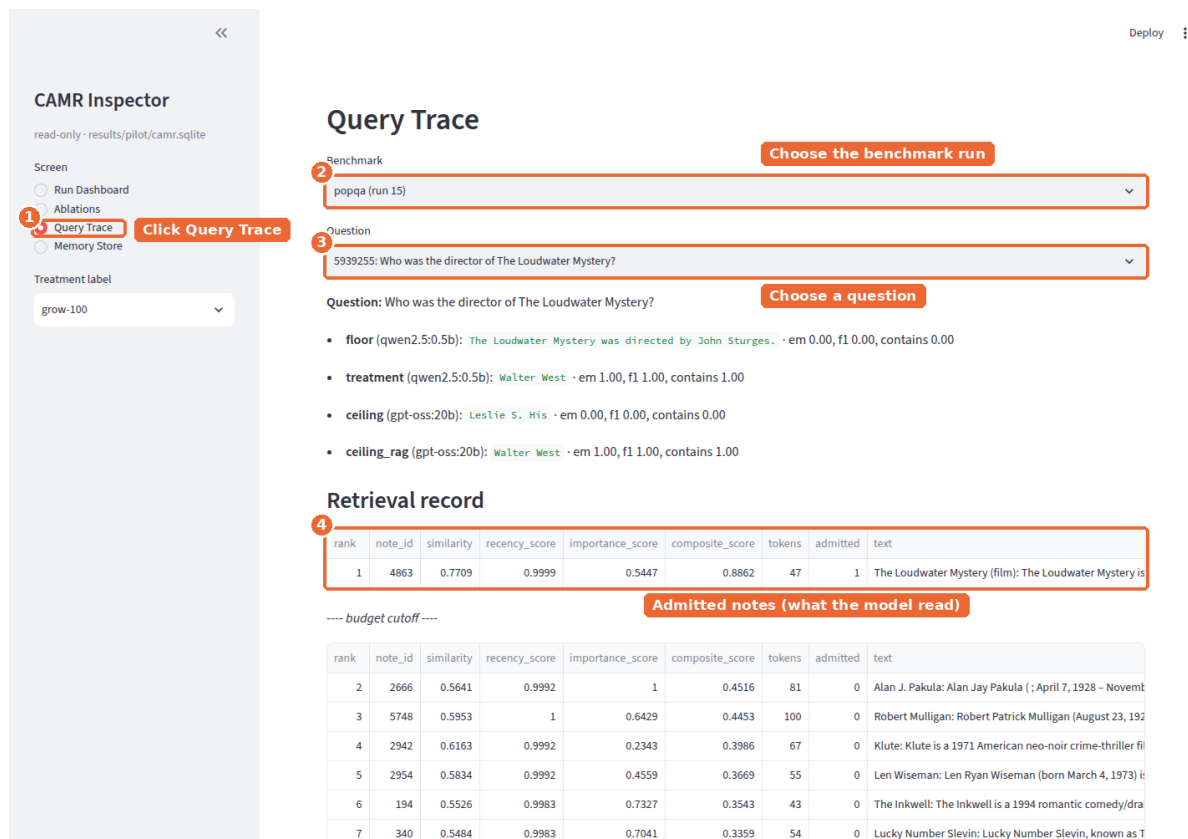


Figure 5.8: Query Trace for one PopQA question at full memory, showing the admitted note and the budget cutoff

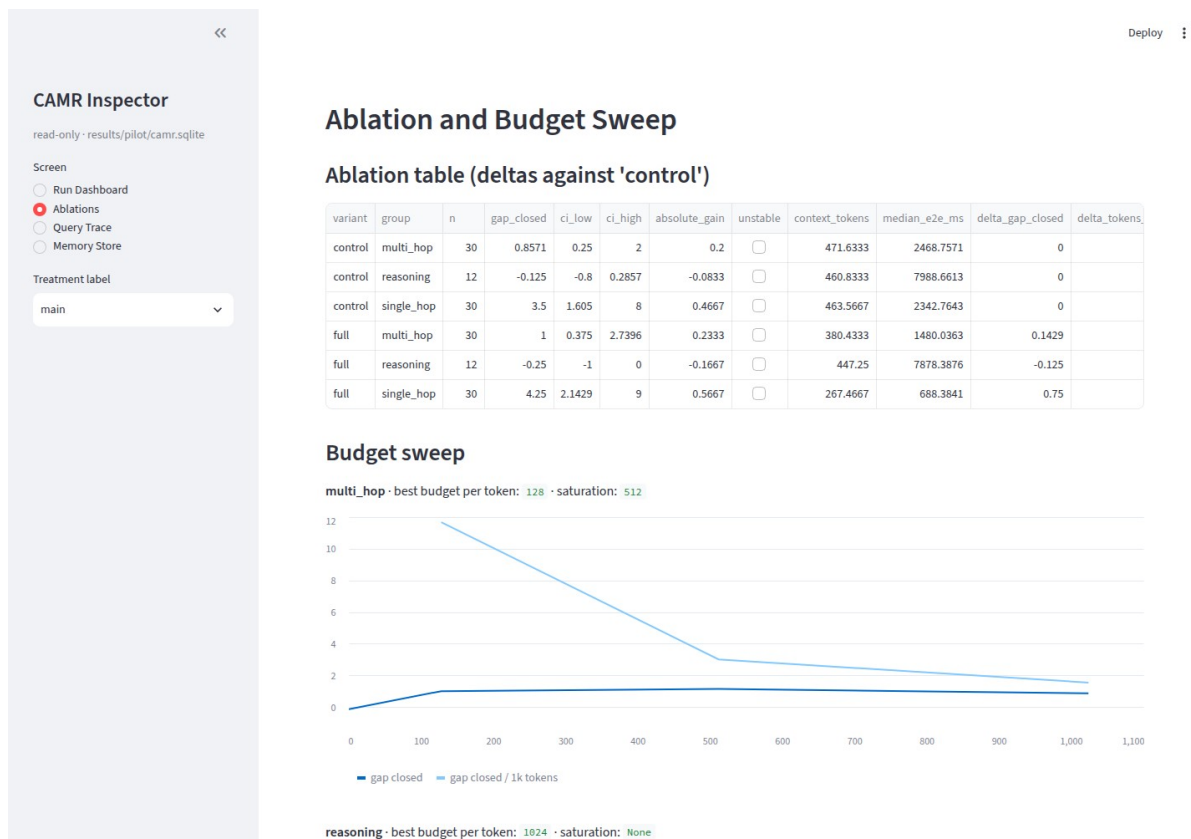


Figure 5.9: Ablations and budget-sweep screen

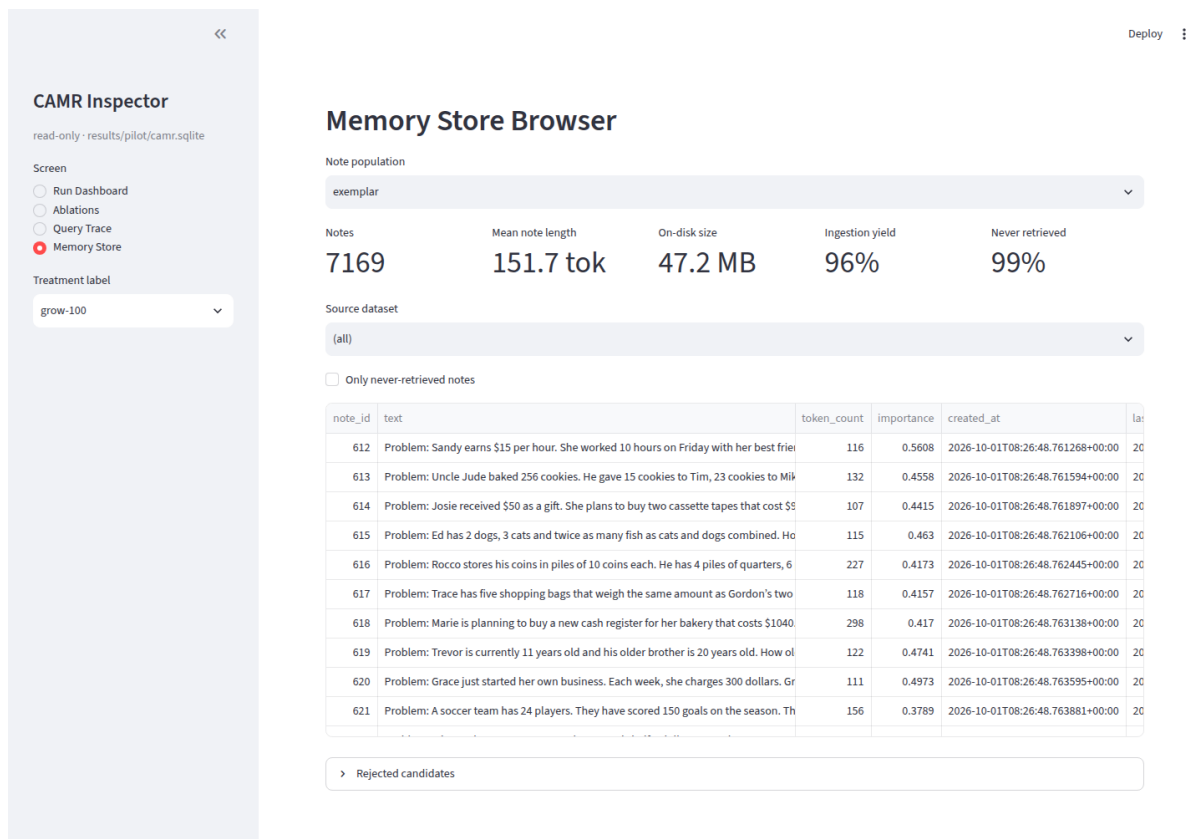
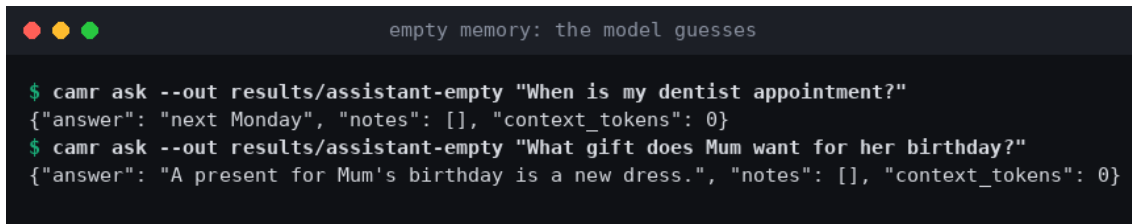


Figure 5.10: Memory Store browser: notes, provenance and retrieval counts

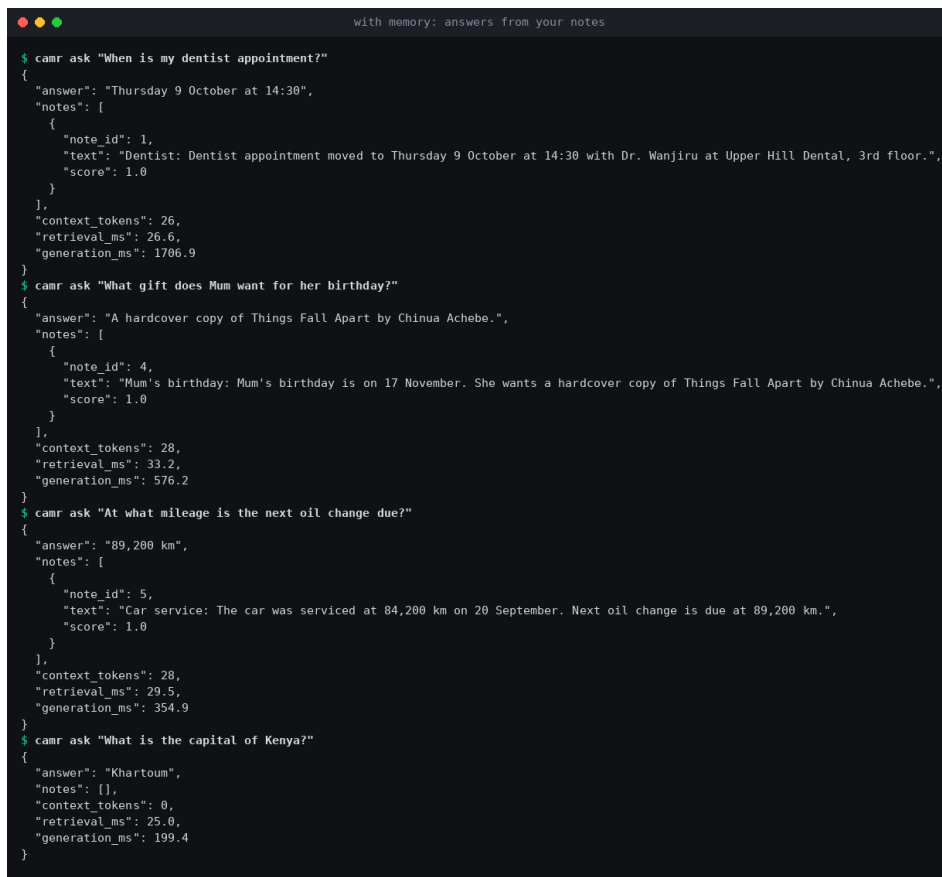
The deployment scenario of section 4.2.1 was exercised with the `camr ask` command on five personal-style notes (Figure 5.11, Figure 5.12). With an empty store, the 0.5B model invented answers ("next Monday"). With the notes, it answered from them ("Thursday 9 October at 14:30"), showing the note used, 26 context tokens, 27 ms retrieval and 1.7 s generation. For an off-topic question the engine abstained (0 notes admitted) and the model answered wrongly from its weights. This failure mode motivates the escalation policy.



```
empty memory: the model guesses

$ camr ask --out results/assistant-empty "When is my dentist appointment?"
{"answer": "next Monday", "notes": [], "context_tokens": 0}
$ camr ask --out results/assistant-empty "What gift does Mum want for her birthday?"
{"answer": "A present for Mum's birthday is a new dress.", "notes": [], "context_tokens": 0}
```

Figure 5.11: Assistant scenario with empty memory: the frozen model guesses



```
with memory: answers from your notes

$ camr ask "When is my dentist appointment?"
{
  "answer": "Thursday 9 October at 14:30",
  "notes": [
    {
      "note_id": 1,
      "text": "Dentist: Dentist appointment moved to Thursday 9 October at 14:30 with Dr. Wanjiru at Upper Hill Dental, 3rd floor.",
      "score": 1.0
    }
  ],
  "context_tokens": 26,
  "retrieval_ms": 26.6,
  "generation_ms": 1706.9
}
$ camr ask "What gift does Mum want for her birthday?"
{
  "answer": "A hardcover copy of Things Fall Apart by Chinua Achebe.",
  "notes": [
    {
      "note_id": 4,
      "text": "Mum's birthday: Mum's birthday is on 17 November. She wants a hardcover copy of Things Fall Apart by Chinua Achebe.",
      "score": 1.0
    }
  ],
  "context_tokens": 28,
  "retrieval_ms": 33.2,
  "generation_ms": 576.2
}
$ camr ask "At what mileage is the next oil change due?"
{
  "answer": "89,200 km",
  "notes": [
    {
      "note_id": 5,
      "text": "Car service: The car was serviced at 84,200 km on 20 September. Next oil change is due at 89,200 km.",
      "score": 1.0
    }
  ],
  "context_tokens": 28,
  "retrieval_ms": 29.5,
  "generation_ms": 354.9
}
$ camr ask "What is the capital of Kenya?"
{
  "answer": "Khartoum",
  "notes": [],
  "context_tokens": 0,
  "retrieval_ms": 25.0,
  "generation_ms": 199.4
}
```

Figure 5.12: Assistant scenario with memory: answers grounded in the user's notes

5.4.7 CAMR Personal: the Engine as an Application

To move from benchmarks to real use, the engine was packaged as **CAMR Personal**, a desktop application that gives any model the user already has in Ollama a long-term memory.

The user teaches it notes and files (.txt, .md, .pdf), dropped anywhere on the window; states facts with *remember that ...*; and approves good answers with a thumbs-up, which stores them as memory. What the user says in chat is also stored, so a fact mentioned weeks earlier is recalled even after it has left the model's context window. Each answer reads only the notes it needs, using the measured best read path (similarity, gating and bridging). A *Recalled N memories* card appears before the answer streams, and the answer shows how grounded it is in those notes. The model's weights never change: the application adapts to its user through storage, not through compute or a longer context window.

The interface was designed after studying the conventions of ChatGPT (a minimal composer and chat history), Ollama's desktop app (the model picker inside the composer, file drop) and Kimi (sources shown before the answer), with memory made the visible centre: a ring that fills as memory grows, and a panel showing what was read, everything stored, and how memory has grown. It is a standard-library Python server with a single-page front end, bound to 127.0.0.1, refusing foreign hosts and cross-site writes. Embeddings come from a small Ollama model (nomic-embed-text), so the download needs no PyTorch. Its abstain threshold (0.55) was calibrated on HotpotQA with the same rule that places BGE's benchmark threshold (0.50): the 90th percentile of off-topic similarity. Self-contained downloads are built by continuous integration for Windows (x64, ARM64), macOS (Apple Silicon, Intel) and Linux (x64, ARM64). Figure 5.13 and Figure 5.14 show the application running with qwen2.5:1.5b on the demonstration notes.

Figure 5.13 traces one message through the application. Numbered screenshots of the main steps follow: first-run setup, the empty chat, a conversation with a remembered fact, a recalled memory and an abstention, the memory panel, and the phone-width layout.

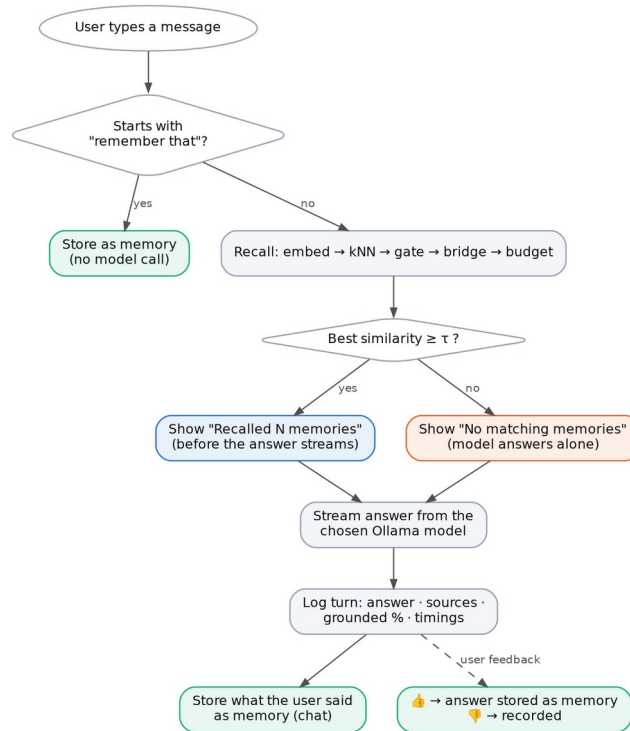


Figure 5.13: One message through CAMR Personal (implemented flow)

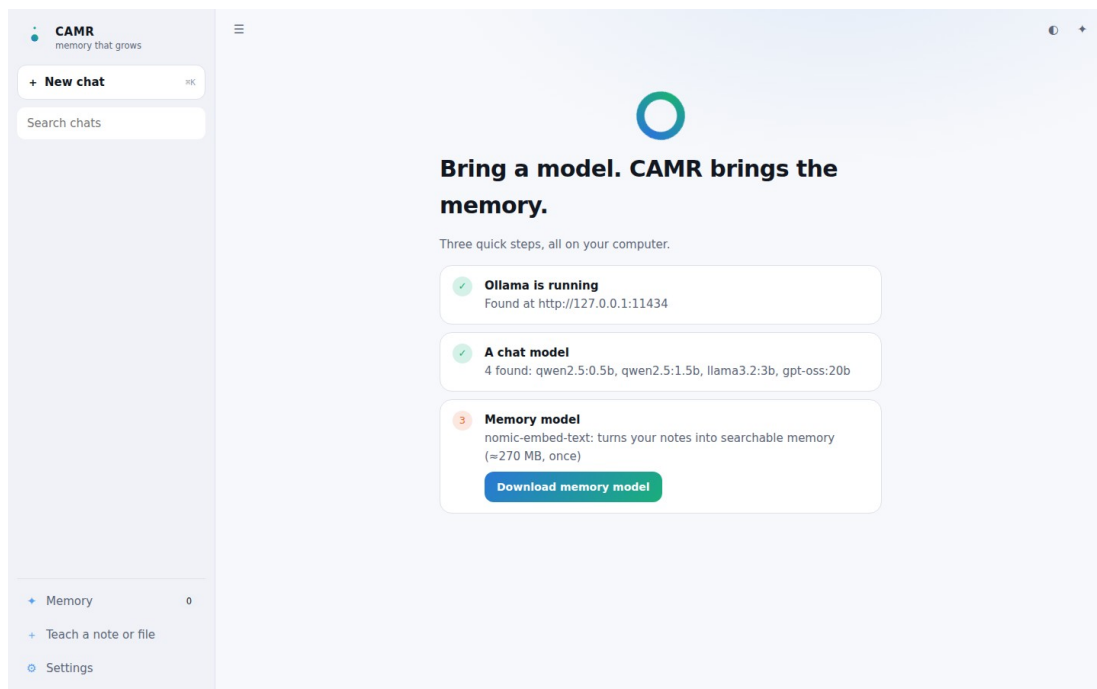


Figure 5.14: Step 1. First run: the app checks Ollama, finds the user's models and downloads the memory model

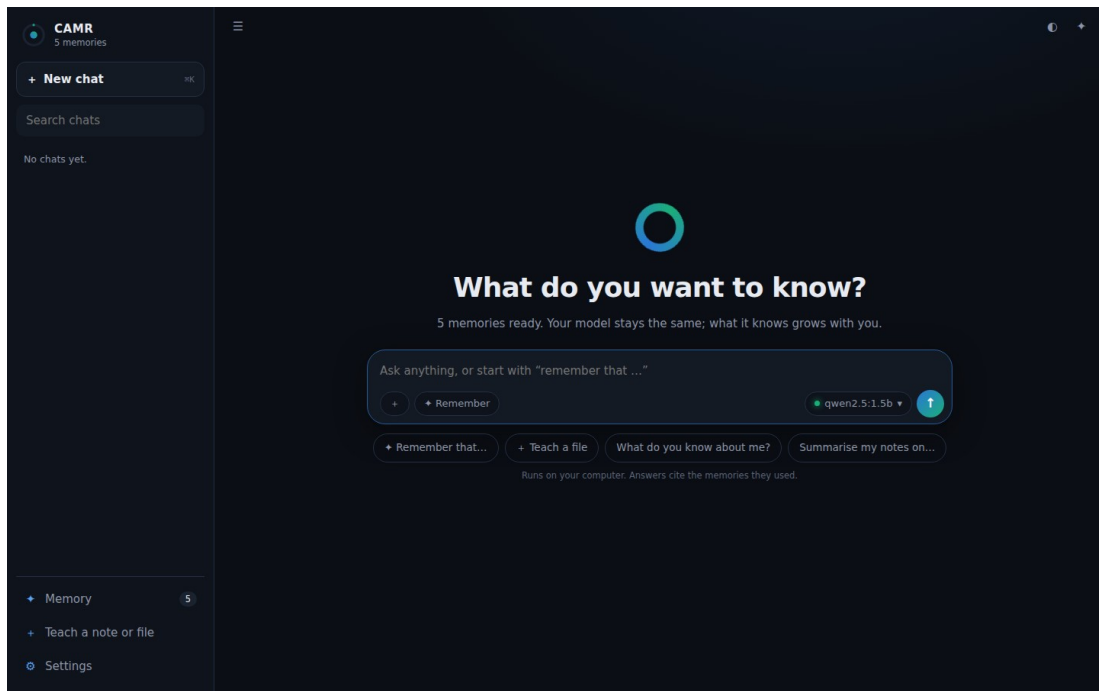


Figure 5.15: Step 2. Empty chat: the model picker sits inside the composer; suggestions start teaching

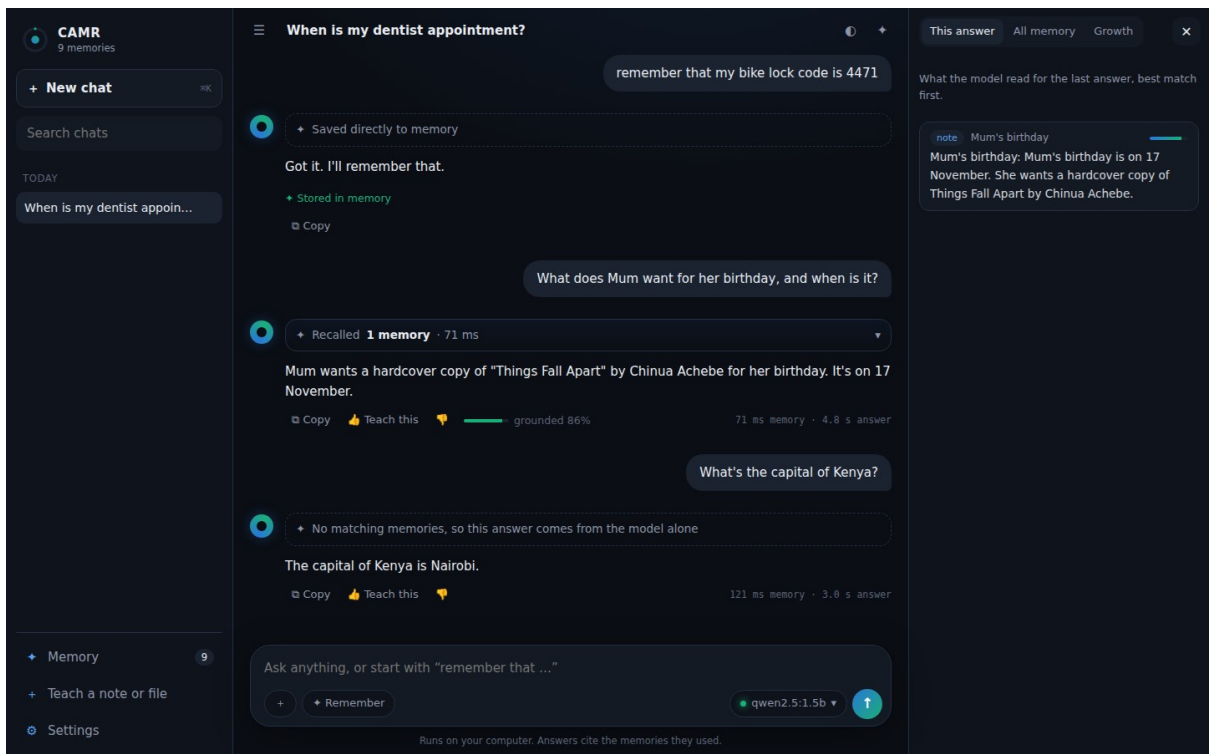


Figure 5.16: CAMR Personal: a remembered fact, an answer with its recalled memory and grounding, and an abstention, with the memory panel open

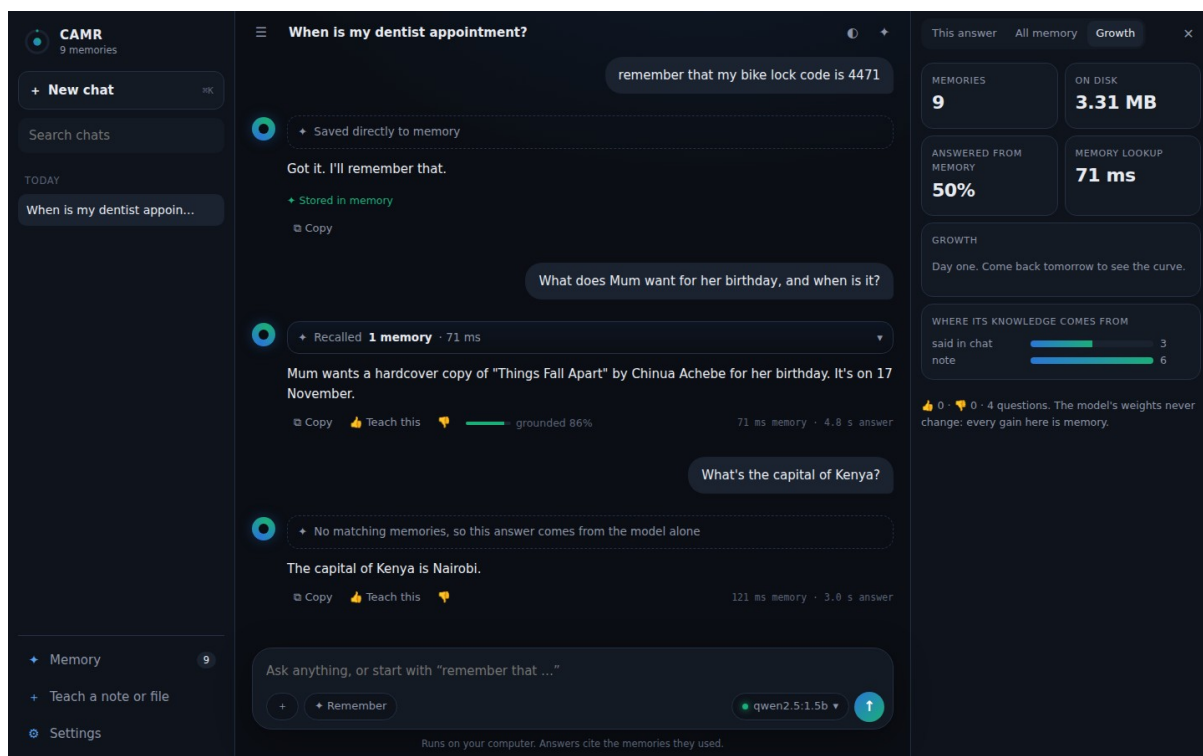


Figure 5.17: CAMR Personal: the growth panel (memories, share of answers drawn from memory, lookup time, sources of knowledge)

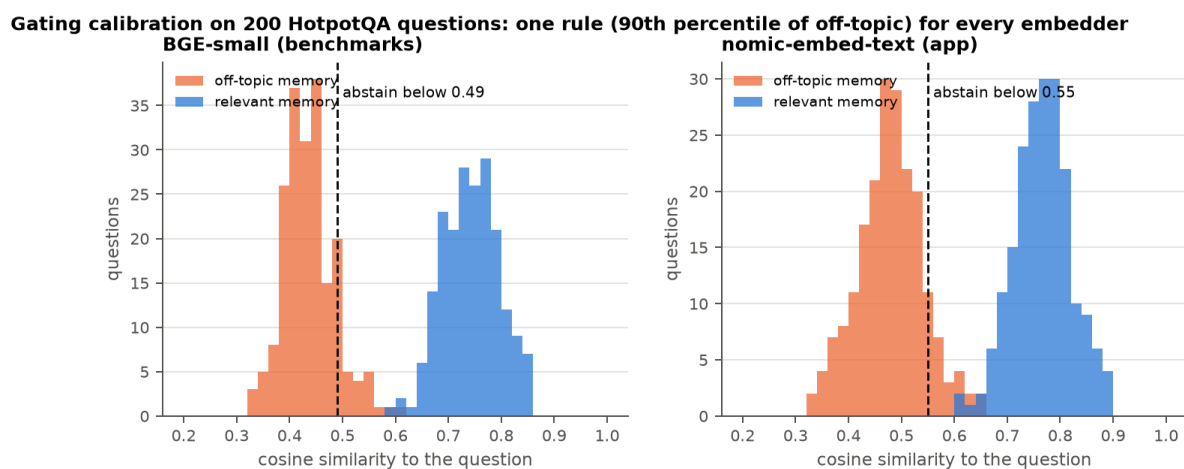


Figure 5.18: Gating calibration: relevant and off-topic similarity on 200 HotpotQA questions; the threshold is the 90th percentile of off-topic similarity for each embedder

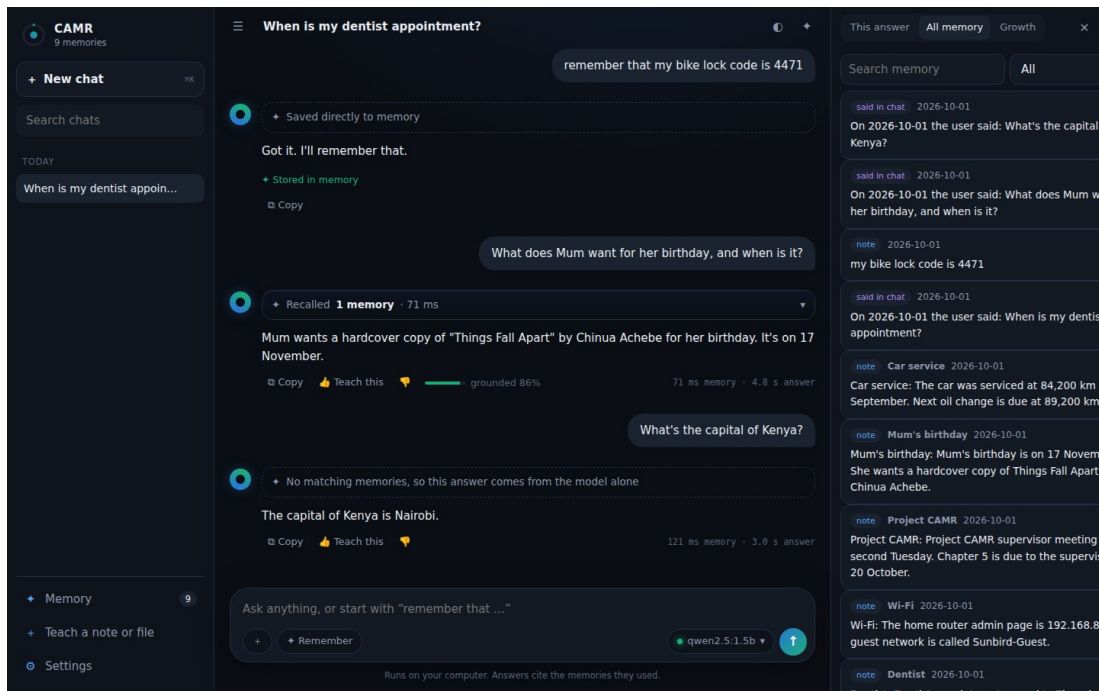


Figure 5.19: All memory: search, provenance by kind, and Forget on every item

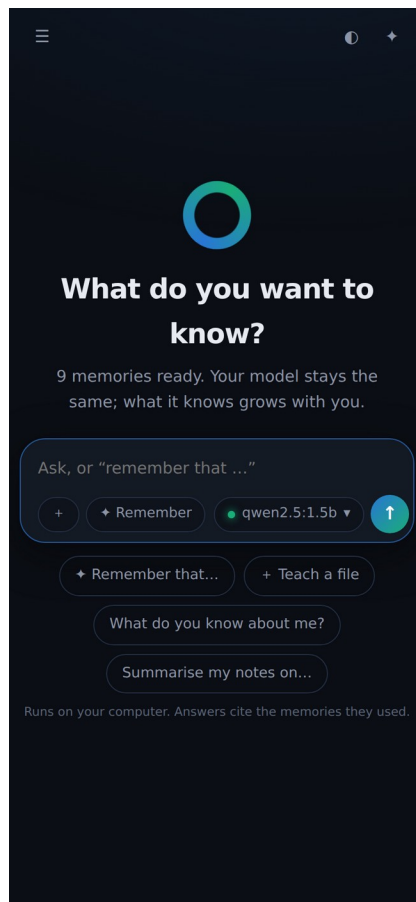


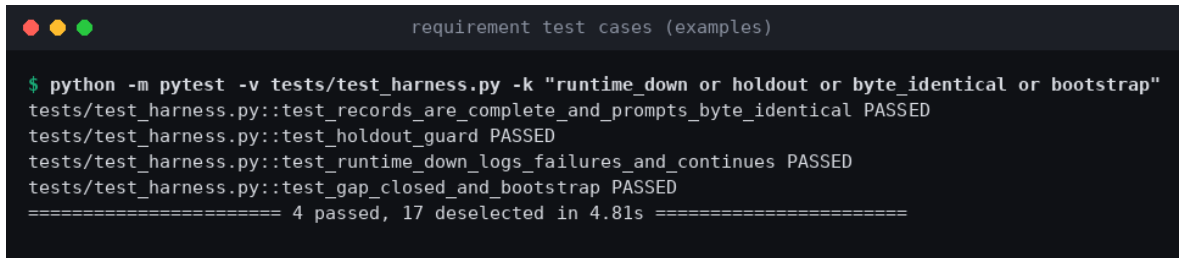
Figure 5.20: The same interface at phone width

5.5 Testing and Evaluation

5.5.1 Test Strategy and Acceptance Criteria

Testing combined three levels. **Unit and integration tests** (white-box, pytest, 105 tests, fully offline) verify each requirement with deterministic test doubles: dry-run models and a hashing embedder. **System tests** run the real pipeline end to end on real data with real models. **Measurements** (device profile, latency, memory) verify the non-functional requirements against their thresholds. Each test maps to a test case of Table 4.5. Acceptance criteria were fixed before the runs: a functional test passes when the observed behaviour equals the expected behaviour; NFR-02 passes when retrieval is $\leq 10\%$ of median end-to-end latency; NFR-05 passes when two runs produce byte-identical prompts.

Figure 5.21 shows a sample of requirement test cases running (captured output).



```
requirement test cases (examples)

$ python -m pytest -v tests/test_harness.py -k "runtime_down or holdout or byte_identical or bootstrap"
tests/test_harness.py::test_records_are_complete_and_prompts_byte_identical PASSED
tests/test_harness.py::test_holdout_guard PASSED
tests/test_harness.py::test_runtime_down_logs_failures_and_continues PASSED
tests/test_harness.py::test_gap_closed_and_bootstrap PASSED
===== 4 passed, 17 deselected in 4.81s =====
```

Figure 5.21: Example requirement test cases: failure handling (IR-07, D-07), holdout guard (DR-04), byte-identical prompts (NFR-05) and the bootstrap (FR-15)

5.5.2 Test Cases, Defects and Retesting

Table 5.7: Test cases and results (unit tests: pytest, 105 passed; measurements: camr profile)

Test ID	Traceability	Scenario and data	Expected result	Actual result and verdict
TC-01	FR-01	Verbatim and structured policies on fixture passages	Notes produced; title kept	As expected. Pass
TC-02	FR-02/03, DR-06	Empty, duplicate, over-length, injection-like notes	Rejected with logged reason; provenance on all notes	As expected. Pass
TC-03	FR-04, IR-03	Commit a note	Timestamp, importance, 384-d embedding stored	As expected. Pass
TC-04	FR-05, IR-04, NFR-03	kNN via sqlite-vec vs NumPy fallback	Identical scores; single file	As expected. Pass
TC-05	FR-06/07	Similarity-only and composite ranking; Eq. 4.3–4.4	Order follows the equations	As expected. Pass
TC-06	FR-08	Budgets 0–1,024 over random note lengths	Budget never exceeded	As expected. Pass
TC-07	FR-09, IR-01	Runner interface; non-loopback host	Only generate; remote host refused	As expected. Pass
TC-08	FR-10, IR-02, NFR-09	Cloud runner with failing transport	Bounded retries, then logged failure	As expected. Pass

Test ID	Traceability	Scenario and data	Expected result	Actual result and verdict
TC-09	FR-11, DR-02	Shuffle the dataset file; redraw	Same sample; persisted with seed and hash	As expected. Pass
TC-10	FR-12	EM, F1, contains, GSM8K final number	Matches hand-computed values	As expected. Pass
TC-11, TC-19	FR-13, NFR-05, NFR-08	Run the same config twice	Complete records; byte-identical prompts	Unit: pass. System: 30/30 identical prompts. 0.5B answers identical on 20/30 only (D-02)
TC-12	FR-14, NFR-07	Ablation plan from config	All variants and budgets planned	As expected. Pass
TC-13	FR-15	Gap closed and bootstrap; near-zero denominator	Correct CI; unstable flagged	As expected. Pass
TC-14	FR-16, NFR-01	Profile on machine A, 3 × 10 questions	Peak memory recorded; fits 16 GB	Engine 712 MB + runtime 1987 MB. Pass
TC-15	FR-17, IR-06	Write through the inspector connection	Write fails at the driver	As expected. Pass
TC-17	NFR-02	Retrieval share of median latency, 0.5B	≤ 10%	56% in the pilot profile. Fail (D-05; see 5.8)
TC-18	NFR-04, IR-05	camr reproduce end to end (dry-run and real)	All tables and figures regenerated	As expected. Pass
TC-20	NFR-06	Block sockets; run floor and treatment	No external connection	As expected. Pass
TC-21	DR-03/04	Corpus containing an evaluation question	Ingestion refused	As expected. Pass
TC-22	IR-07	Local runtime down mid-run	Failures logged; run continues; store intact	As expected. Pass
TC-23	Deployment (4.2.1)	CAMR Personal: teach, ask, remember, 👍, forget, settings, export; server API, streaming, host/origin checks	Answers cite notes; chat recalled beyond the window; export hides text; foreign hosts refused	14 tests and a browser test with a real model. Pass

The defects below were found during the real runs. Each was fixed and retested, or is reported as an open limitation. Failed results were kept in the record, not deleted.

Table 5.8: Defects found and retests

ID	Defect	Found by	Correction	Retest
D-01	The screener rejected the real song title "I Can't Get Next to You" as a model refusal	Retrieval run 1 rejection log	Refusal check applied only to model-generated notes	Regression test added. Pass
D-02	The 0.5B model's answers differed on 10/30 questions between identical runs (all wrong → wrong)	Repeat runs	Engine confirmed deterministic (prompts identical); floors reported as a range	Open: runtime non-determinism
D-03	gpt-oss:20b was killed for running out of memory at 13.3 GB	Pilot ceiling run	Memory-mapped weights; one model loaded at a time	Loaded. Pass
D-04	Changing a configuration field mid-study changed the hash, so resume re-ran finished runs	Pilot resume	Freeze the schema before final runs	Pass
D-05	Retrieval took 56% of answer time (NFR-02)	TC-17	Diagnosed as CPU contention; thread partitioning settings added	Engine alone 29 ms; p90 157 → 50 ms. Still above 10% for 0.5B

ID	Defect	Found by	Correction	Retest
D-06	Composite score (Eq. 4.1) lowered full-support recall by 4.6 points	Retrieval evaluation	Similarity + gating + bridging as the best read path	2Wiki recall 35.8% → 78.4%; HotpotQA equal at 11% fewer tokens
D-07	When the model server stopped mid-sweep, failed queries were logged (IR-07) but the run was still marked completed, so resume skipped it	Machine B sweep	Resume re-runs any run with failed queries; the failed run is kept	Regression test added. Pass; 19 runs re-run clean
D-08	gemma2:9b was killed for running out of memory three times at 13.4 GB (runtime prompt cache growing on top of the weights)	Machine B sweep	Runtime prompt cache capped at 1 GiB (affects prefill reuse only)	No further kills. Pass
D-09	CAMR Personal: the shared store connection failed when the web UI called it from another thread, and the UI's file watcher crashed while crawling the PyTorch modules	App browser test	Store opened for cross-thread use behind a lock; file watcher disabled	App tests and browser test. Pass
D-10	CAMR Personal: the new warm-up loaded the model with the default context size, so Ollama loaded it a second time for the first question (1.8 s to the first word instead of a fraction of a second)	App latency benchmark	Warm-up sends the same options as a chat request	Regression test; latency re-measured (Table 5.20). Pass

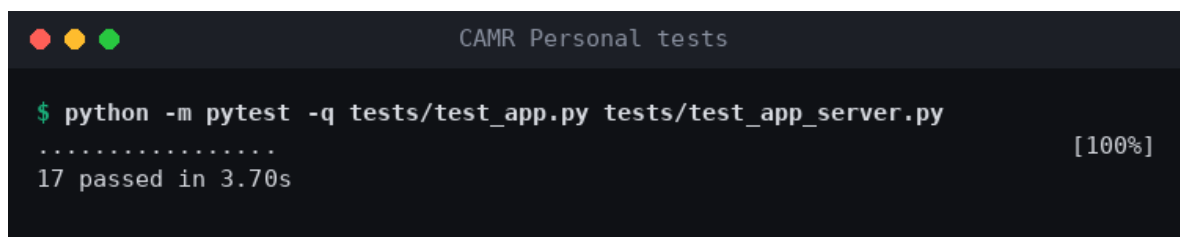
5.5.3 Application Testing (CAMR Personal)

The application was tested as a user would meet it, with valid, empty, out-of-scope and unsupported input and with the runtime missing. The live checks used qwen2.5:1.5b and nomic-embed-text through Ollama in a browser driven by Playwright. The automated checks are in `tests/test_app*.py` and in the continuous-integration smoke test.

Table 5.9: Application test cases and results

Test ID	Scenario	Expected result	Actual result and verdict
TC-24	Empty message (spaces only)	Rejected; no model call; no chat created	Error event, no chat (test). Pass
TC-25	Out-of-scope question ("What's the capital of Kenya?") with only personal notes stored	Gating abstains; the answer is labelled as coming from the model alone	0 memories admitted; labelled; answered "Nairobi". Pass
TC-26	"remember that my bike lock code is 4471"	Stored without a model call; duplicate reported	Stored; second time "already in memory" (test and browser). Pass
TC-27	Question answered by a note	Recalled memory shown before the answer; grounded answer	1 memory, 71 ms; correct answer, grounded 86%, 4.8 s (1.5B, CPU). Pass
TC-28	Fact said in chat, then asked after it left the context window	Recalled from memory	Recalled as "something you said" (test). Pass
TC-29	Teach a Markdown file and a PDF	Chunked and stored as memory	Stored (test; PDF via pypdf). Pass
TC-30	Ollama not running / memory model missing	Setup screen with the missing step and a download button	Setup screen; download with progress; then chat (browser; CI checks the not-running state).

Test ID	Scenario	Expected result	Actual result and verdict
			Pass
TC-31	Request with a foreign Host header, or a cross-site POST	Refused (403)	403 (test). Pass
TC-32	Path traversal (../../etc/passwd)	No file outside the app is served	Index page served (test). Pass
TC-33	Downloadable build on six platforms	Builds, launches, serves the UI and API	All six green in CI (Table 5.11). Pass
TC-34	Time to first word recorded for every answer	Measured from prompt sent to first streamed word; never larger than the answer time	Recorded and shown under the answer (test). Pass
TC-35	App starts, or the user switches model	Chat model loaded in the background before the first question	Empty-prompt load request sent (test); first answer faster (Table 5.20). Pass
TC-36	Same question asked twice	Second lookup reuses the question's embedding; documents never served from the cache	One embedding call for two lookups (test). Pass



```

CAMR Personal tests

$ python -m pytest -q tests/test_app.py tests/test_app_server.py
..... [100%]
17 passed in 3.70s

```

Figure 5.22: Application test run: tests for the assistant core and the web server (captured output)

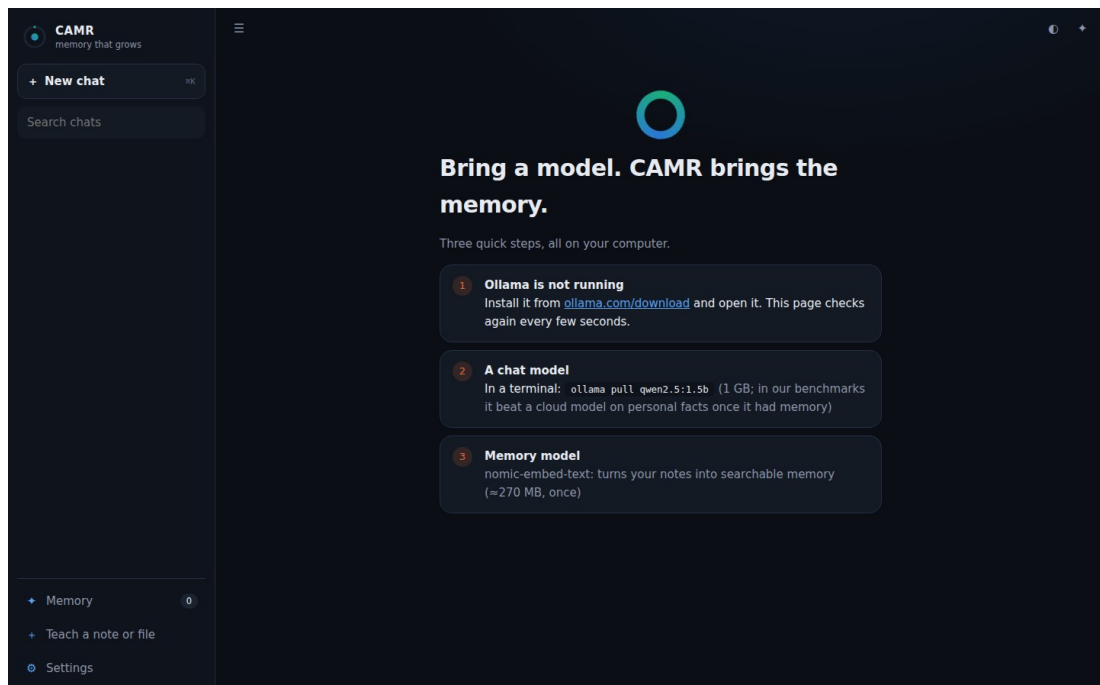


Figure 5.23: TC-30: with Ollama not running, the app shows what is missing instead of failing

These tests show that the application behaves correctly and fails safely. They do not show that people find it useful or easy to use. That requires a user study, which had not been run when this chapter was written (see Limitations, section 5.8).

5.5.4 Evaluation Design

The primary measure is the fraction of the capability gap closed (Equation 3.1) per task type, together with the absolute gain, the residual gap, tokens per question and latency. Two ceilings were used: the local gpt-oss:20b and the cloud Kimi K3. Against a closed-book ceiling that the small model plus memory *exceeds*, gap closed rises above 1 and stops being a fraction. Kimi K3, the frontier reference, is therefore the anchor for headline statements. With $n = 30$, one question is 3.3 percentage points, and confidence intervals are reported where they matter.

5.6 Version Control and Implementation Evidence

Project repository: <https://github.com/Satasdin/CAMR> (development branch claude/engine-build-docs-yfw9u0, merged through a pull request into main). The repository contains the README, an illustrated setup walkthrough (docs/SETUP.md), the findings log with every measurement and its source file (docs/FINDINGS.md), the design critique (docs/DESIGN_NOTES.md), dependency files (pyproject.toml), every experiment configuration (configs/), the test suite, and the scripts that regenerate every figure and screenshot in this chapter. Raw results databases and downloaded datasets are excluded by .gitignore, because of their size and licence terms, and are regenerated by the scripts. The Kimi API key was supplied only through an environment variable and does not appear in any commit.

Table 5.10 lists the commit history as recorded by Git. Each commit corresponds to an implemented or measured increment, and the commit messages name the finding they add. The model-size sweep ran on separate machines on result branches, whose databases were merged with `camr merge`.

Table 5.10: Commit history (git log)

Commit	Date	Author	Change
5fe043f	29 Sep 2026 09:03	Satasdin	Build the CAMR memory engine and evaluation harness from the proposal
66fa3de	29 Sep 2026 09:03	Satasdin	Initial commit
797ad01	01 Oct 2026 06:58	Satasdin	Make CAMR capability-adaptive and measure the residual gap
9a7e1b3	01 Oct 2026 07:38	Satasdin	Add real-model pilot tooling, retrieval evaluation and screening fix

Commit	Date	Author	Change
3700f6a	01 Oct 2026 07:57	Satasdin	Add memory-growth experiment, larger comparison models and 2Wiki
6f9d0d0	01 Oct 2026 08:16	Satasdin	Add model-size sweep, cross-machine merge and data fetch script
f9d557e	01 Oct 2026 08:29	Satasdin	Add OpenAI-compatible cloud ceiling (Kimi) and second-ceiling analysis
a746641	01 Oct 2026 09:08	Satasdin	Findings log with real pilot results, figures and inspector screenshots
1eec1bd	01 Oct 2026 09:08	Satasdin	Add Kimi K2.6 as a second cloud ceiling config
d738a34	01 Oct 2026 09:44	Satasdin	Learn the engine's per-question policy from rewards (contextual bandit)
a30ba07	01 Oct 2026 10:30	Satasdin	Findings: full pilot results (residual gap, ablation, budget sweep, profile), new screenshots, registry metadata
15206ae	01 Oct 2026 10:40	Satasdin	Findings F9: small model + CAMR vs Kimi K2.6/K3 with and without memory
06fed37	01 Oct 2026 10:51	Satasdin	Findings F10: model-size sweep (machine E) with figure; sweep figure readability fixes
6e9211c	01 Oct 2026 11:24	Satasdin	Findings F11-F12: learned engine policy (contextual bandit) and retrieval-latency diagnosis
e619530	01 Oct 2026 11:26	Satasdin	Grounding cascade: answer locally, escalate ungrounded answers; beats always-Kimi (65.3% vs 63.9%) with 29% cloud
0c90bd3	01 Oct 2026 11:27	Satasdin	Findings F10: add machine C models (gemma3:1b, smollm2, granite3.3:2b, falcon3:3b, phi4-mini)
1bf38e0	01 Oct 2026 11:27	Satasdin	Findings: correct count of models reaching Kimi on PopQA (10 of 11)
ea52fce	01 Oct 2026 11:38	Satasdin	Setup walkthrough with annotated screenshots and a personal-notes assistant demo
1ff3ff8	01 Oct 2026 11:44	Satasdin	Chapter 5 builder: proposal Word styles, results read from the results files
bbb65f2	01 Oct 2026 11:47	Satasdin	Findings F10b: machine B (7-14B) merged; fix D-07 resume after runtime failure
945327b	01 Oct 2026 11:48	Satasdin	Chapter 5 builder: column widths, per-model answer time, defects D-07/D-08
376eed2	01 Oct 2026 11:50	Satasdin	Chapter 5: AI-use declaration
e42a555	01 Oct 2026 12:24	Satasdin	Findings: machine D merged (19 models) and 2WikiMultiHopQA retrieval
e76a716	01 Oct 2026 12:28	Satasdin	CAMR Personal: a downloadable local assistant whose memory grows with use
e810501	01 Oct 2026 12:29	Satasdin	Chapter 5 builder: 2Wiki retrieval table, 19-model discussion, app section, n=200 study; hotpot200 config and retrieval figures
fd89571	01 Oct 2026 12:37	Satasdin	MIT licence; remove the Discord launch kit
f2d4da1	01 Oct 2026 15:37	Ibrahim Sudheysi Ali	Merge CAMR engine, evaluation harness, findings and CAMR Personal app
08062d3	01 Oct 2026 13:34	Satasdin	Android prototype (parked): Kotlin port of the CAMR engine with a Compose UI
48505a0	01 Oct 2026 13:34	Satasdin	CAMR Personal 0.2: custom chat UI, Ollama embeddings, desktop downloads
2f278c3	01 Oct 2026 13:38	Satasdin	App screenshots (dark/light/mobile), user guide for the downloads, F13 n=200 results, Chapter 5 updates
1775be6	01 Oct 2026 16:42	Ibrahim Sudheysi Ali	Merge CAMR Personal 0.2: new chat UI, desktop downloads, n=200 confirmation
1aa5b23	01 Oct 2026 13:43	Satasdin	Release workflow: publish a release from a manual dispatch; reject empty chat messages

Commit	Date	Author	Change
99a23a4	01 Oct 2026 13:47	Satasdin	Chapter 5: before/after data sample, model selection protocol, error analysis, application tests, CI evidence, desktop-to-mobile progress; report PDF and DOCX
0ecac1b	01 Oct 2026 14:09	Satasdin	Report: architecture/pipeline/flow diagrams, model rankings, n=200 and error charts, test screenshots, reproduction steps; Chapter 6 builder
83cb804	01 Oct 2026 14:10	Satasdin	Chapters 5 and 6: final build with both gating calibrations
aa77465	01 Oct 2026 14:32	Satasdin	App speed: model warm-up with chat options (D-10), time to first word, query-embedding cache; latency benchmark; Chapter 5/6 restructure (alignment with Chapter 1, Android to future work); viva Q&A
a749573	01 Oct 2026 14:52	Satasdin	App latency results (F14): warm-up 2.7-3.4x faster first answer, query cache, memory-size lookup; Chapters 5 and 6 rebuilt
23af328	01 Oct 2026 14:52	Satasdin	Track the app latency results; drop the LibreOffice profile from the repo
1af615d	01 Oct 2026 17:53	Ibrahim Sudheysi Ali	Merge: app speed, latency results, Chapters 5 and 6, viva Q&A
51e573a	01 Oct 2026 17:24	Satasdin	CAMR Personal 0.2.1: background model warm-up, time to first word, query-embedding cache
6fc811b	01 Oct 2026 20:29	Ibrahim Sudheysi Ali	Merge: CAMR Personal 0.2.1

The work followed GitHub flow. Each round of work was developed on one branch and merged through a pull request whose description summarises the change and its test plan: Satasdin/CAMR#2 (engine, harness, findings), Satasdin/CAMR#3 (CAMR Personal 0.2), Satasdin/CAMR#4 (application speed, latency results and report) and Satasdin/CAMR#5 (CAMR Personal 0.2.1). Machines B–E pushed their results to their own branches, which were merged into the results database rather than into the code. Feedback from users is collected through a GitHub issue form ([.github/ISSUE_TEMPLATE/feedback.yml](#)). The code is released under the MIT licence. Continuous integration ([.github/workflows/release-app.yml](#)) runs the application tests and builds and launch-tests the downloadable application on six platforms for every change to the application. Table 5.11 shows the run for the merged version. Published releases (v0.2.0, then v0.2.1 with the speed changes) attach the six downloads.

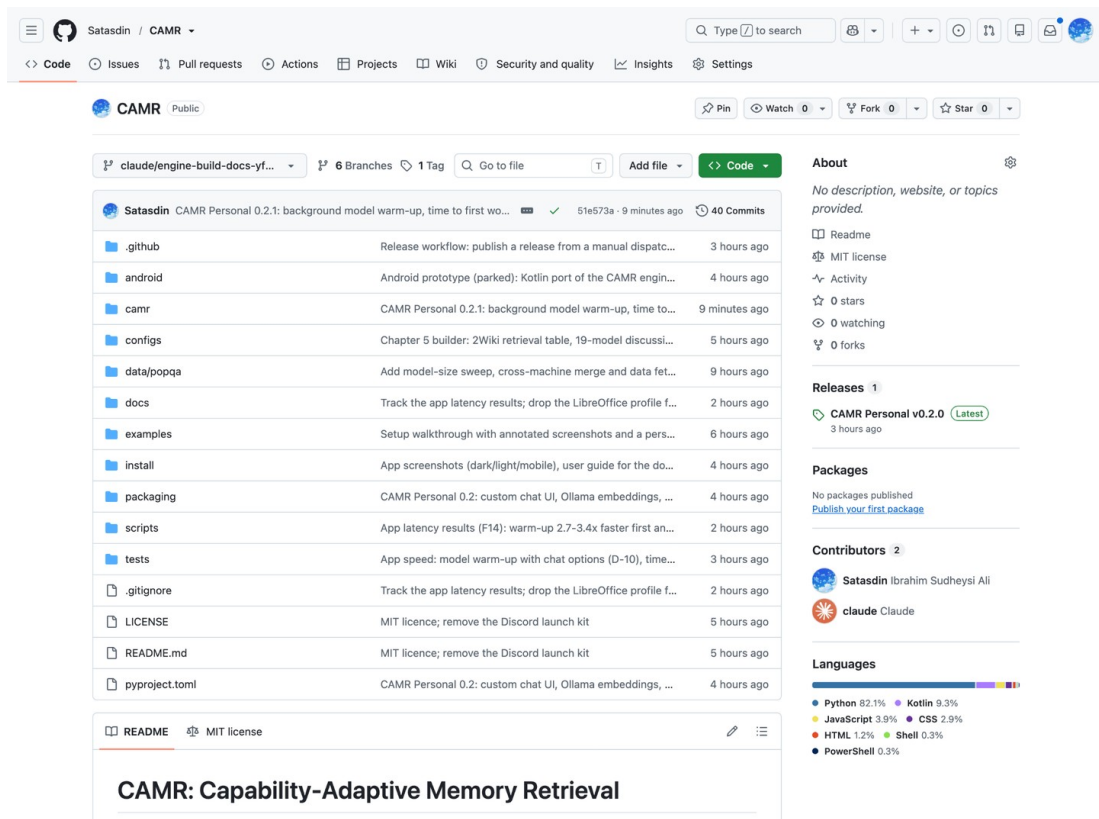


Figure 5.24: Project repository on GitHub (README, MIT licence, releases, contributors) (screenshot)

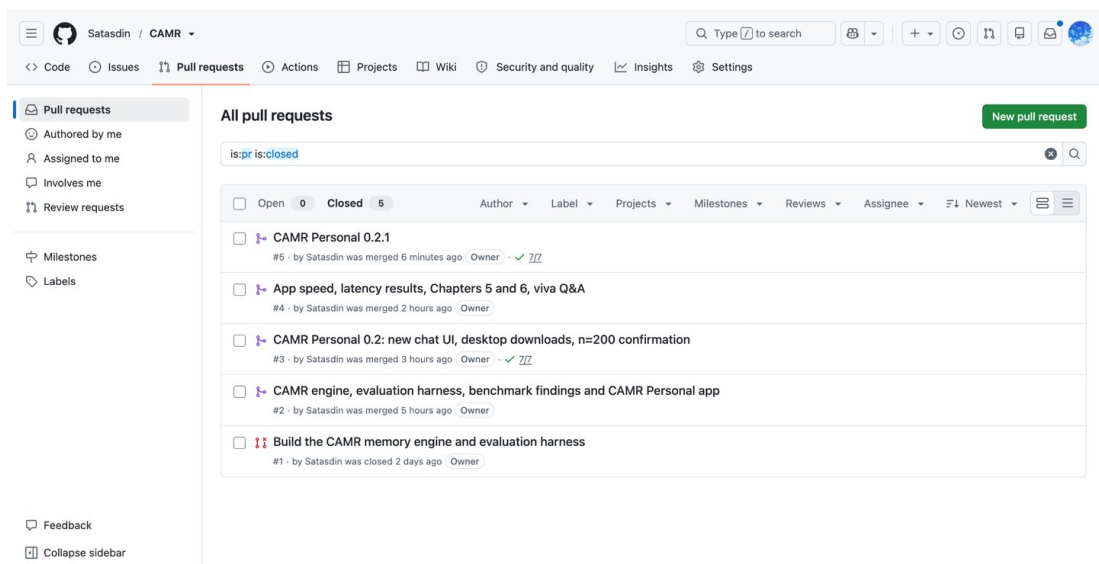


Figure 5.25: Pull requests used to merge each round of work (screenshot)

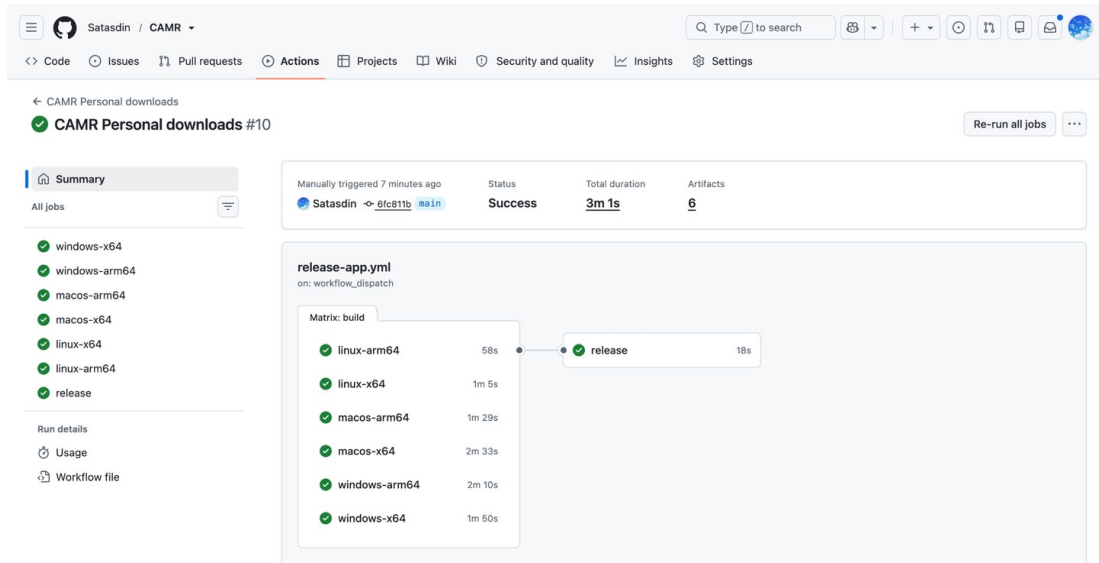


Figure 5.26: Continuous-integration run: tests, build and launch test on six platforms (screenshot)

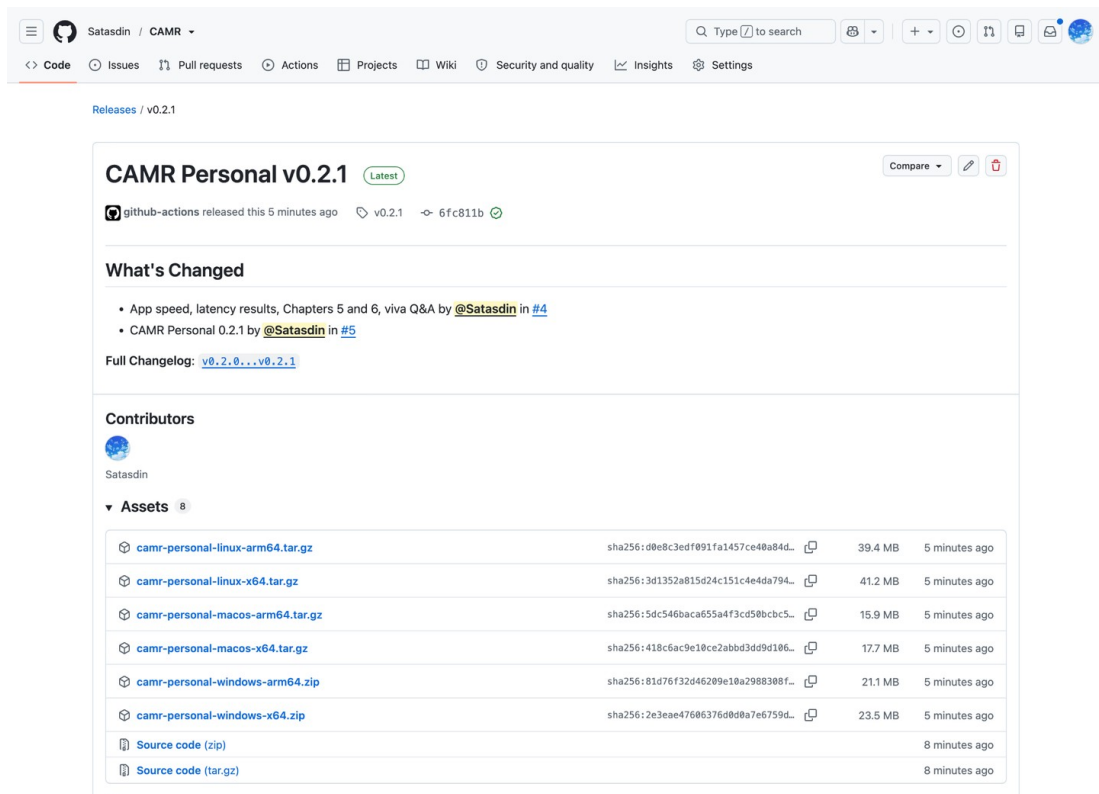


Figure 5.27: Release v0.2.1 with the six downloadable files and their checksums (screenshot)

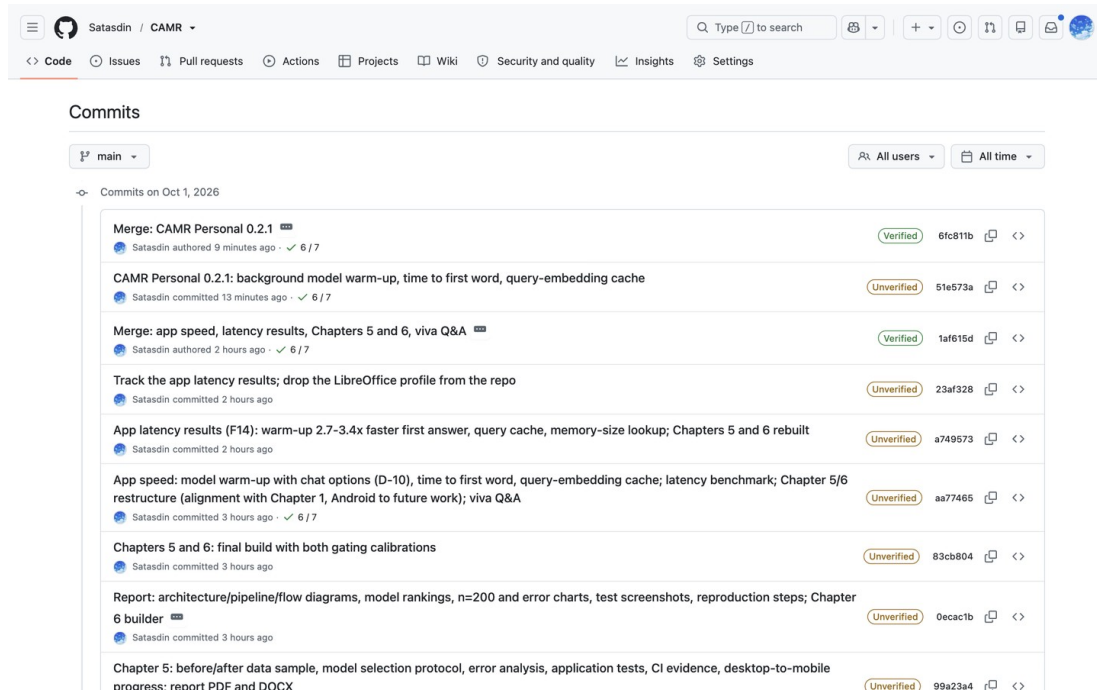


Figure 5.28: Commit history on the main branch (screenshot)

Table 5.11: Continuous-integration evidence: build and launch test per platform (GitHub Actions run 36899712846, release v0.2.1)

Target	Runner	Tests	Build	Launch test	Result
Windows x64	windows-2025	pass	pass	pass	success
Windows ARM64	windows-11-arm	pass	pass	pass	success
macOS Apple Silicon	macos-15	pass	pass	pass	success
macOS Intel	macos-15-intel	pass	pass	pass	success
Linux x64	ubuntu-24.04	pass	pass	pass	success
Linux ARM64	ubuntu-24.04-arm	pass	pass	pass	success

5.7 Author Oversight and Use of AI-Assisted Tools

Claude Code, an AI coding assistant provided by Anthropic, was used throughout the implementation and evaluation stages of this project. Under the student's direction it assisted with writing the engine and harness code and its tests, running the benchmark experiments on the cloud machines, producing the figures and screenshots, and drafting this chapter from the logged results. The student defined the research problem, objectives, requirements and design (Chapters 1–4), supplied the cloud model access, and decided which experiments were run, including the comparison with cloud models, the wider model-size sweep, the learned escalation policy and the use of real rather than synthetic benchmark data.

The student reviewed the intermediate results and screenshots as the work progressed and redirected it where it fell short; most importantly, the student required that this chapter report only experiments that had actually been run and benchmarked with real models, and an earlier draft written before the results existed was discarded for that reason. The AI-assisted output was checked through the automated test suite (105 tests), the per-query logs, the read-only inspector, and the findings log, which names the results file behind every reported number, so that each claim can be traced and re-run. The student made the final decisions and takes full responsibility for the submitted work.

5.8 Evaluation Results, Discussion and Limitations

The results are organised by research objective. Source files are in `results/` and every table is reproduced in `docs/FINDINGS.md`.

5.8.1 Retrieval Quality (Objective ii)

Before any language model was involved, the read path was evaluated on 500 HotpotQA and 500 2WikiMultiHopQA questions against one store of 38,054 notes built from 23,524 real paragraphs (170 MB). The measure was whether *both* gold supporting paragraphs reached the context, because a multi-hop question cannot be answered from one of them.

Table 5.12: Full-support recall and tokens supplied, 500 questions each
(`results/retrieval_eval/tables/retrieval_eval.json`)

Read path (512 tokens unless noted)	2Wiki: both gold	Tokens	HotpotQA: both gold	Tokens
Similarity only (baseline)	35.8%	471	70.8%	470
+ recency and importance (Eq. 4.1)	34.6%	470	65.4%	467
+ gating	33.4%	408	64.6%	384
+ entity bridging, no gating	75.2%	472	70.0%	470
Similarity + gating + bridging	78.4%	439	70.8%	420
Baseline @ 1,024 tokens	41.4%	980	79.8%	978
Similarity + gating + bridging @ 1,024	84.0%	821	83.0%	728

HotpotQA (multi-hop): retrieval quality and cost against the similarity-only baseline (n=500)

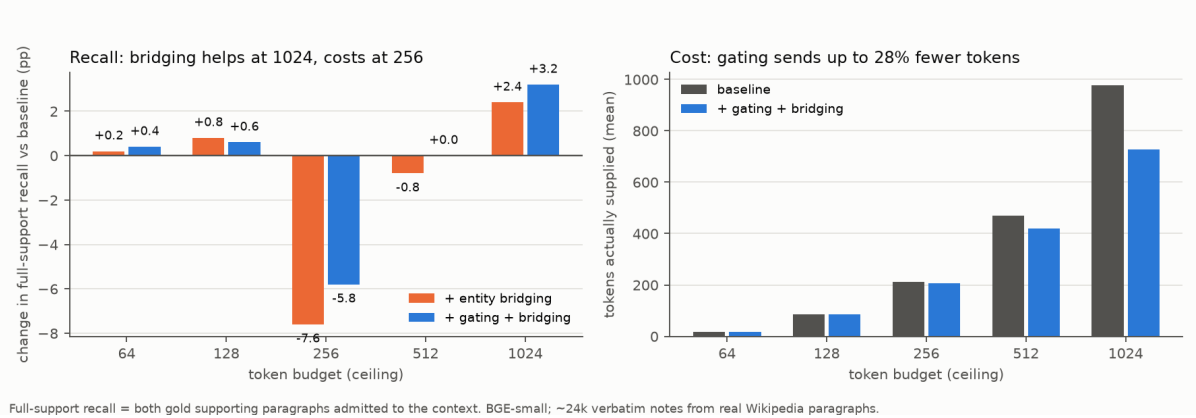


Figure 5.29: HotpotQA: change in full-support recall and tokens supplied relative to the baseline

Entity bridging was the decisive mechanism on 2WikiMultiHopQA: full-support recall rose from 35.8% to 78.4% at the same budget, with 7% fewer tokens. Its questions are compositional (for example, *who is the father of the director of film X?*). The second paragraph shares almost no words with the question, so similarity search cannot find it, but the first paragraph names it, and that is the hop bridging follows. On HotpotQA the gain depended on the budget: none at 512 tokens, with 11% fewer tokens, and +3.2 points at 1,024 tokens, with 26% fewer tokens. On both benchmarks the composite score of Equation 4.1 lowered recall. On independent questions, recency and importance carry no signal about the current question, which confirms the risk anticipated in section 4.4.7.

5.8.2 Gap Closed and the Residual Gap (Objective iv)

Table 5.13: Accuracy by condition, pilot sample (results/pilot/tables/summary.json, gap_vs_kimi.md)

Task (n)	0.5B alone	0.5B + CAMR	gpt-oss-20b	20B + same notes	Kimi K3	Kimi K3 + CAMR
Single-hop, PopQA (30)	10.0%	66.7%	23.3%	76.7%	66.7%	83.3%
Multi-hop, HotpotQA (30)	6.7%	30.0%	30.0%	53.3%	53.3%	63.3%
Reasoning, GSM8K (12)	33.3%	16.7%	100%	100%	100%	100%

Against Kimi K3, the engine closed the **entire** single-hop gap (gap closed 1.00) and **half** of the multi-hop gap (0.50, 95% CI 0.20–0.90). It closed none of the reasoning gap (−0.25). The 0.5B model with CAMR answered single-hop questions in 1.5 s against Kimi's 7.7 s. The residual gap (20B with the same notes minus 0.5B with them) was 10.0 points on single-hop, 23.3 on multi-hop and 83.3 on reasoning: the more a task needs reading and reasoning, the

more model size still buys. This answers research question iii. Memory helps knowledge-bound tasks and does not help reasoning-bound ones. The same notes also lifted the frontier model (Kimi K3 rose from 66.7% to 83.3% on PopQA), so the engine helps beside any model.

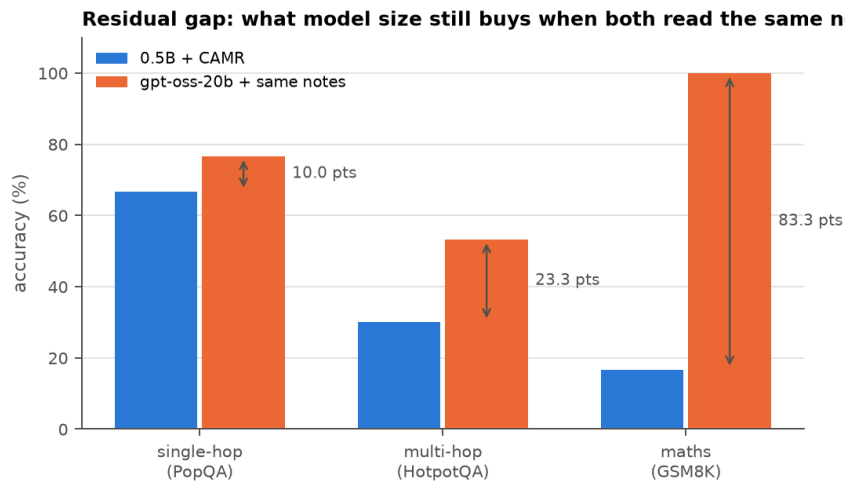


Figure 5.30: Residual gap: accuracy of the 0.5B model and of gpt-oss-20b when both read the same notes (pilot)

5.8.3 Error Analysis: Where Answers Are Lost

Table 5.14 follows each question through the pipeline (from `tables/coverage.md`): is the gold answer in the store, among the retrieved candidates, in the context actually sent, and was the answer correct? The last column is accuracy *given* that the answer was in the context.

Table 5.14: Failure decomposition for the 0.5B model with the full engine (pilot)

Benchmark (n)	Answer in store	In candidates	In context	Correct	Correct when in context
PopQA (30)	93.3%	93.3%	90.0%	66.7%	74.1%
PopQA, 128-token budget (30)	93.3%	93.3%	86.7%	76.7%	84.6%
HotpotQA (30)	96.7%	93.3%	83.3%	30.0%	36.0%

Retrieval was rarely the problem. On HotpotQA the answer reached the context for 83% of questions, but the 0.5B model used it correctly only 36% of the time. Of its 21 misses, 16 (76%) had the gold answer in front of it. On PopQA a smaller budget raised correct-when-in-context from 74% to 85%: fewer notes, less distraction. Errors therefore fall into three groups, illustrated in Table 5.16: **correct** answers read from the note; **incomplete** answers that pick a related entity from the right evidence; and **unsupported** answers, where the model ignored or lacked the evidence and fell back on its own (wrong) knowledge.

Where answers are lost (0.5B + CAMR, pilot): retrieval delivers; the small model's reading is the bottleneck
PopQA (single-hop) **HotpotQA (multi-hop)**

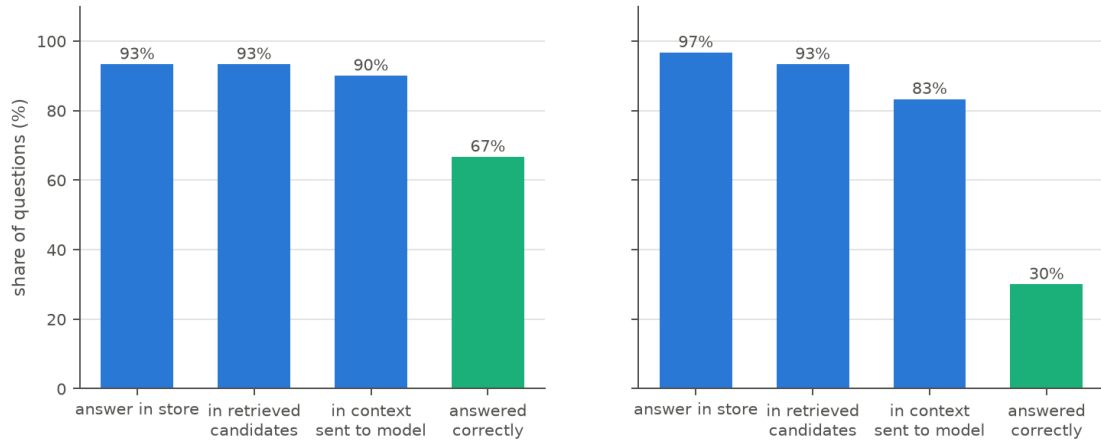


Figure 5.31: Where answers are lost for the 0.5B model with CAMR: store, candidates, context and correct answer

Table 5.15: Examples of correct, incomplete and unsupported answers (from the query logs)

Type	Question (abridged)	Gold	Answer given
Correct	Who was the screenwriter for <i>The Graduate</i> ?	Calder Willingham, Buck Henry	0.5B + CAMR: Buck Henry (alone; Linda Wachowski)
Correct	McLaren MP4/11 was driven by which Finnish driver?	Mika Häkkinen	0.5B + CAMR: Mika Häkkinen (gpt-oss-20b alone: Kimi Räikkönen)
Incomplete	City with the Nusretiye Clock Tower and ...?	Istanbul, Turkey	Beyoğlu (a district of Istanbul; evidence was in context)
Incomplete	Event where Tyson Gay and Rodney Martin both represented the US?	4 × 100 m relay	100 m sprint (evidence in context)
Unsupported	Who was the director of <i>The Loudwater Mystery</i> ? (no memory)	Walter West	0.5B alone: John Sturges (invented)
Unsupported	When is my dentist appointment? (empty memory)	Thursday 9 October 14:30	next Monday (invented); with the note: correct

5.8.4 Knowledge Growth with Frozen Weights

The knowledge corpus was fed into one store in three stages, and the same frozen 0.5B model answered the same questions after each stage (Figure 5.32). On long-tail facts, accuracy rose from 6.7% to 16.7%, 33.3% and 66.7% as memory grew to 25%, 50% and 100%. That matches Kimi K3 and is roughly three times the 20B model. Median decode speed stayed between 44 and 63 tokens/s at every stage (Figure 5.33). The cost of memory is in prefill (270–460 extra prompt tokens) and a bounded 3–17% per-token decode slowdown, measured across the model sweep, from attending over a longer prompt.

A frozen 0.5B model gains as its memory grows: on long-tail facts it reaches Kimi K3's score

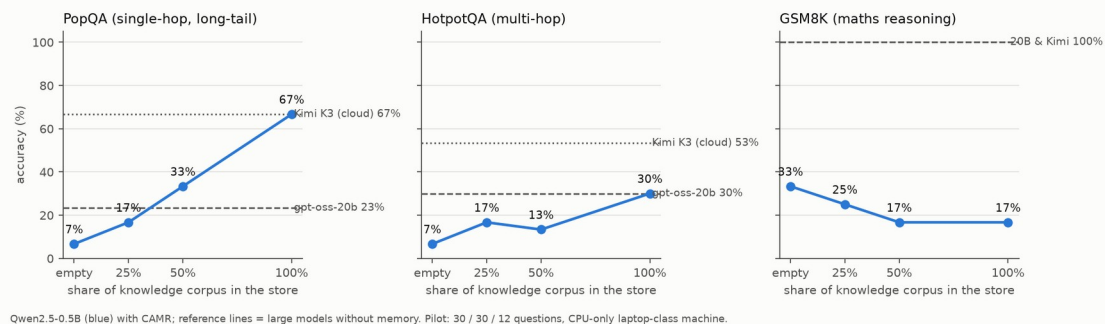


Figure 5.32: Accuracy of the frozen 0.5B model as the memory store grows

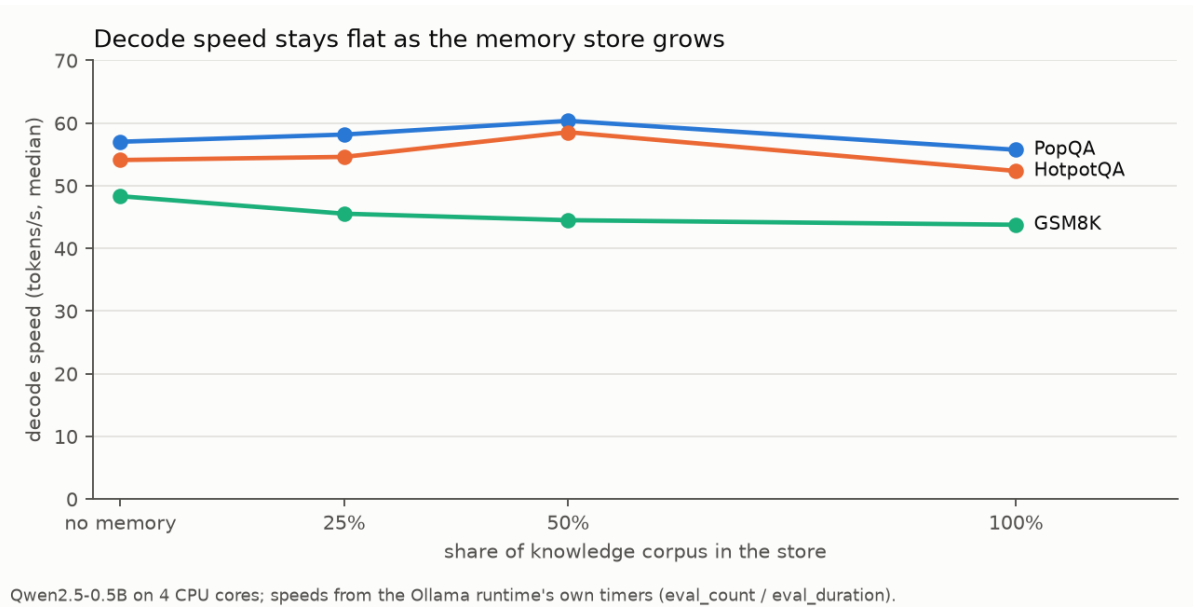


Figure 5.33: Decode speed of the frozen 0.5B model at each store size

5.8.5 Ablation and Token Budget (Objective v)

Table 5.16: Ablation and budget sweep, 0.5B model (ablation.md, budget_sweep.md)

Variant	Single-hop acc. (tokens)	Multi-hop acc. (tokens)	Reasoning acc.	Answer time (single / multi)
Control: similarity only, fixed budget	56.7% (464)	26.7% (472)	—	2,343 / 2,469 ms
Full engine: gating + bridging	66.7% (267)	30.0% (380)	—	688 / 1,480 ms
Budget 0	10.0% (0)	3.3% (0)	16.7%	—
Budget 128	76.7% (82)	30.0% (86)	16.7%	—
Budget 512	66.7% (267)	33.3% (380)	20.8%	—
Budget 1,024	66.7% (434)	26.7% (557)	16.7%	—

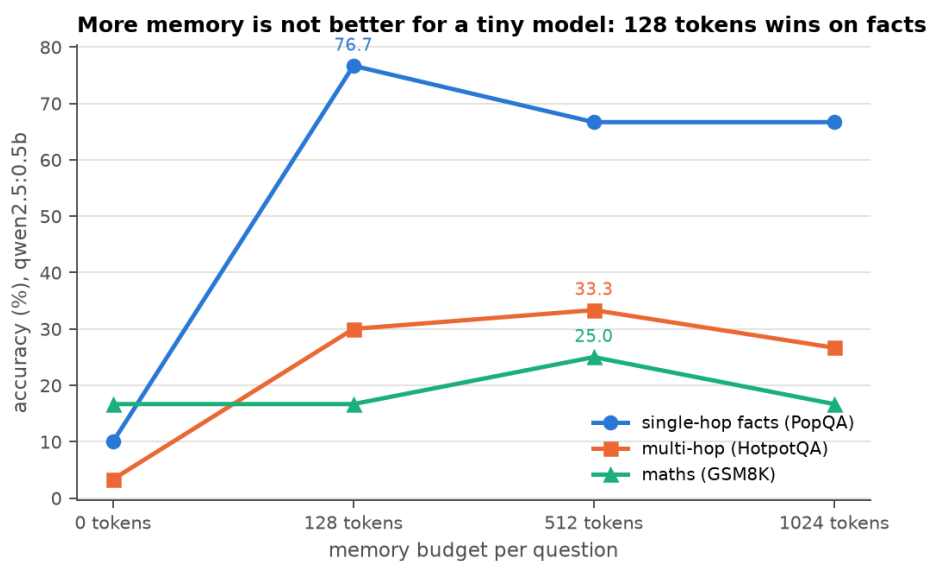


Figure 5.34: Accuracy of the 0.5B model by memory budget (pilot)

The full engine was more accurate than the control and 1.7–3.4 times faster per answer, because gating sent fewer tokens. For a 0.5B model, the budget that maximised gap closed per token was **128 tokens** for single-hop facts. Larger budgets *lowered* accuracy, because the small model was distracted by extra notes. Multi-hop questions peaked at 512 tokens, and reasoning never benefited. This answers research question iv: the best budget depends on the task type and the model, which motivated the learned policy.

5.8.6 Small Models on Their Own: the Baseline Ranking

Before memory, the 19 candidate models were ranked by their own accuracy (Figure 5.35). The ranking follows size, but loosely: the rank correlation between parameter count and mean accuracy was $\rho = 0.82$. gemma3:12b led (51.7% mean), followed by qwen2.5:7b (47.8%) and gemma2:9b (47.2%); qwen2.5:0.5b was last (13.9%). The models' weakness was knowledge, not arithmetic. Alone, no model knew more than 33.3% of the long-tail facts (Kimi K3 knew 66.7%), and the mean across all 19 was 17.7%. Several, however, already scored 75–92% on GSM8K. Multi-hop questions were also out of reach: the best was qwen2.5:14b at 46.7%, below Kimi K3's 53.3%. Model families differed markedly at the same size. qwen2.5:3b was 16th of 19 on its own (6.7% on facts) while llama3.2:3b was 8th, and mistral:7b was 14th, below several 1–3B models.

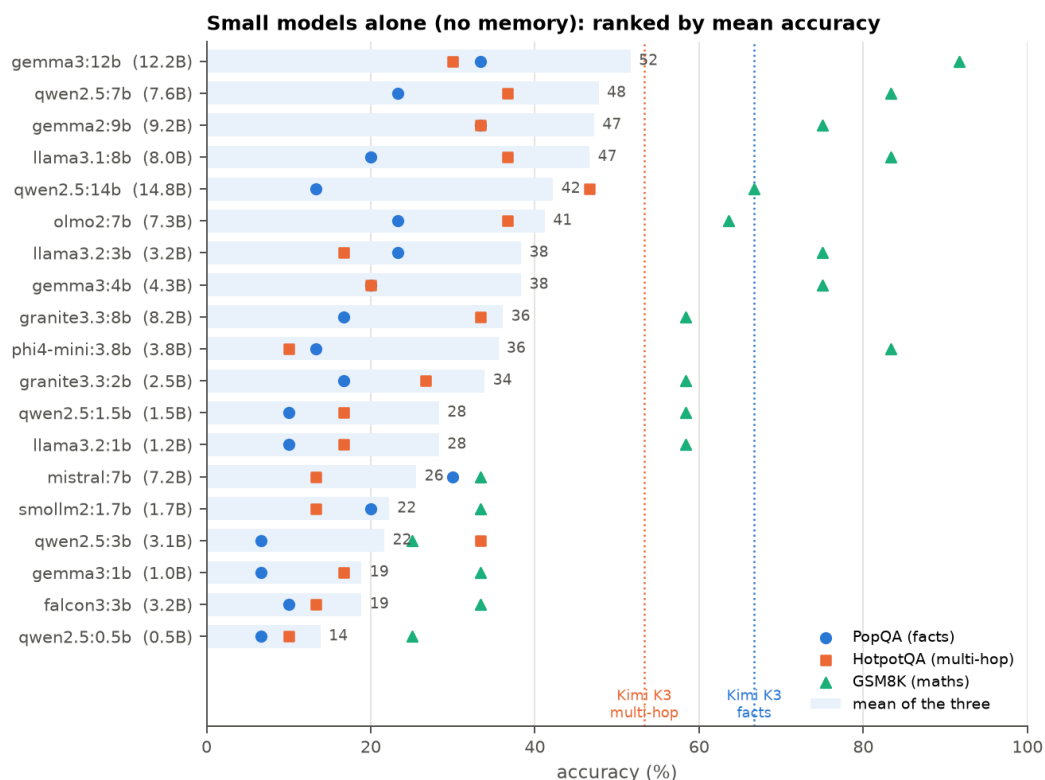


Figure 5.35: Ranking of the 19 small models alone (no memory): mean accuracy and per-benchmark scores, with Kimi K3 for reference

With CAMR the ranking changed (Figure 5.36). The size advantage on facts largely disappeared: the rank correlation between size and long-tail accuracy fell from 0.67 ($p = 0.002$) alone to 0.35 ($p = 0.14$, not significant) with memory, and mean fact accuracy rose from 17.7% to 74.2%. The models that gained most climbed the ranking: phi4-mini moved from 10th to 5th, falcon3:3b from 17th to 10th, and qwen2.5:14b from 5th to 2nd. Models with strong parametric knowledge but weaker reading of supplied evidence fell: gemma2:9b from 3rd to 8th, llama3.1:8b from 4th to 9th. Multi-hop accuracy with memory still tracked size ($p = 0.78$), because combining two pieces of evidence depends on the model as a reader. Figure 5.37 shows each model's gain and Figure 5.38 its speed.

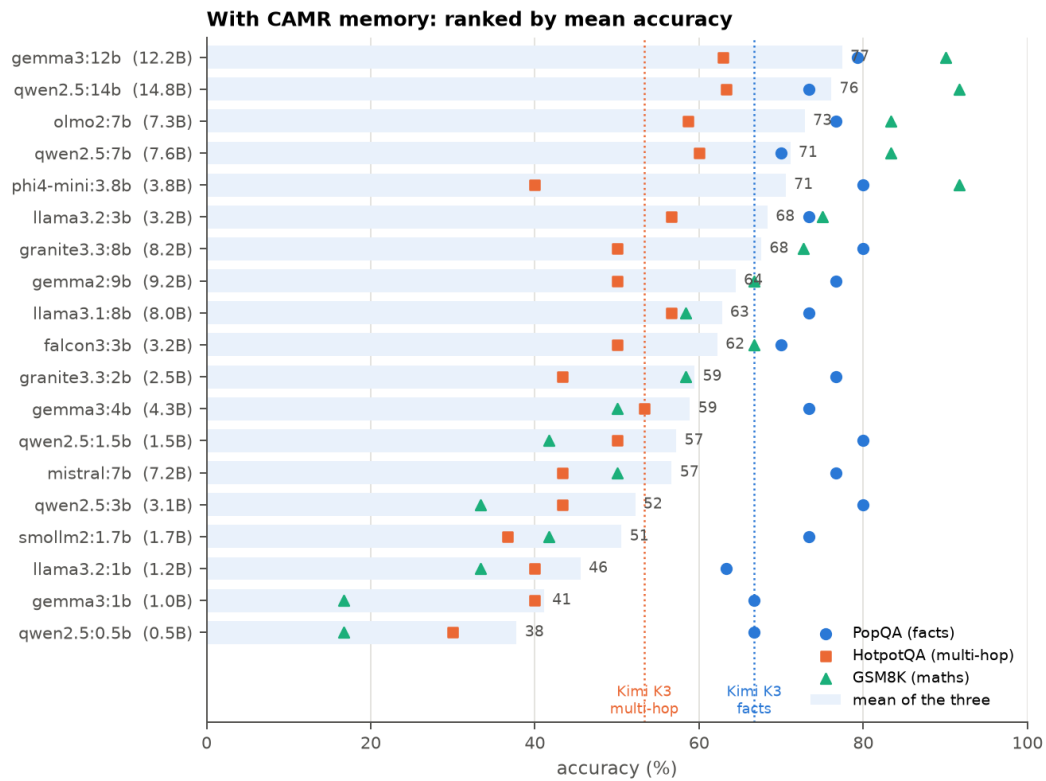


Figure 5.36: Ranking of the same 19 models with CAMR memory

What memory adds, per model (percentage points; blue = helps, orange = hurts)

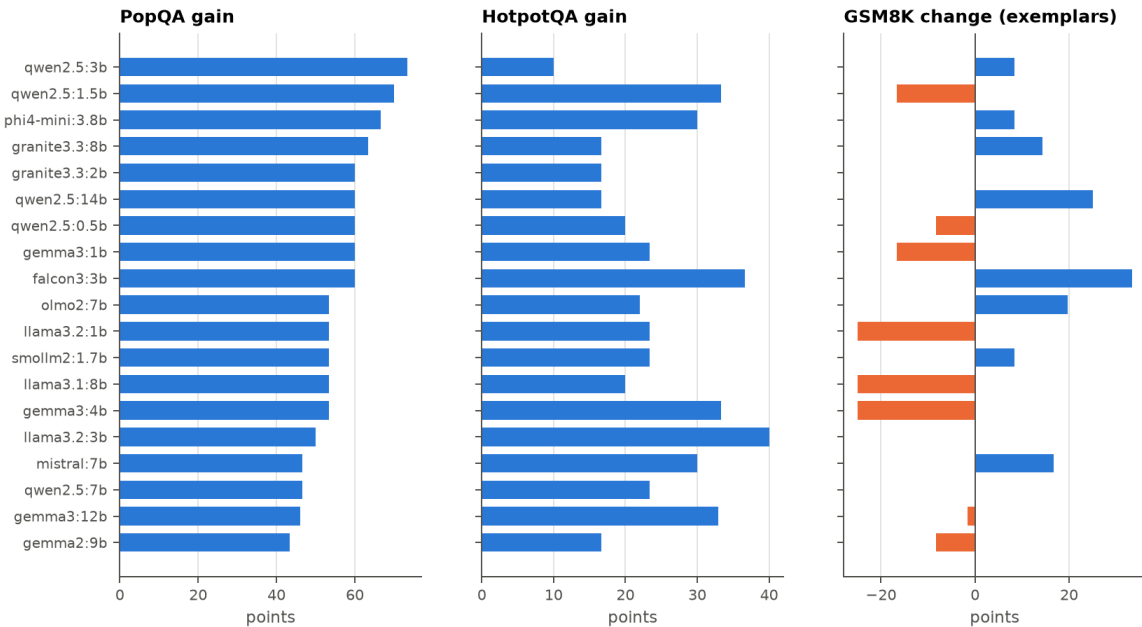


Figure 5.37: What memory adds to each model, per benchmark (percentage points)

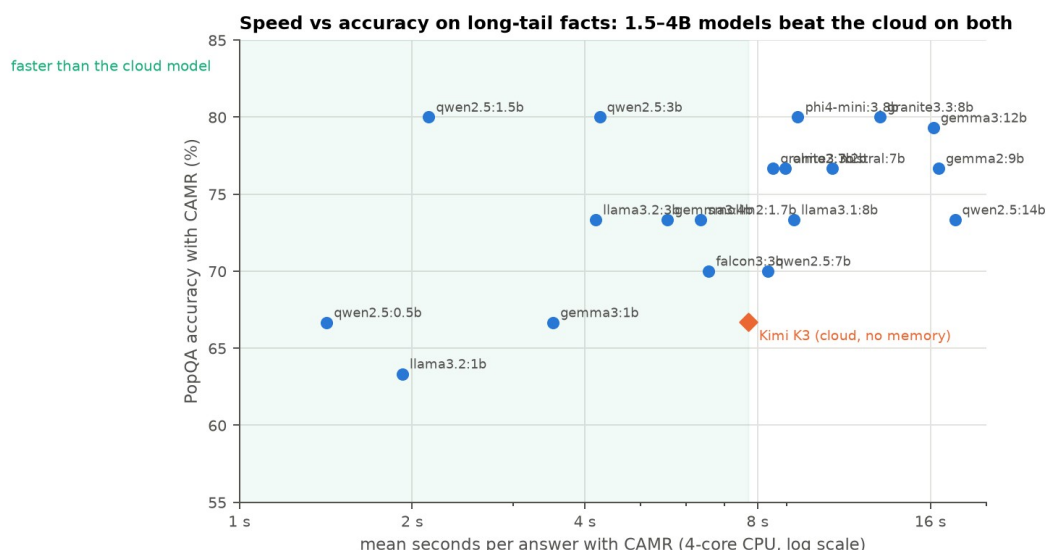


Figure 5.38: Speed against accuracy with CAMR on long-tail facts; the shaded region is faster than the cloud model

5.8.7 Which Model Sizes Suit the Engine

The same store, questions and ceilings were used with 19 small models from 0.5B to 14.8B parameters (Table 5.17, Figure 5.39). Accuracies are comparable across machines; decode speeds are comparable only within a machine.

Table 5.17: Model-size sweep: alone → with CAMR (results/pilot/tables/model_sweep.json)

Model (parameters, quantisation)	PopQA (%)	HotpotQA (%)	GSM8K (%)	Decode tok/s (PopQA)	s / answer with CAMR (PopQA)
qwen2.5:0.5b (494.03M, Q4_K_M)	6.7 → 66.7	10.0 → 30.0	25.0 → 16.7	50.3 → 48.9	1.4
gemma3:1b (999.89M, Q4_K_M)	6.7 → 66.7	16.7 → 40.0	33.3 → 16.7	24.9 → 23.9	3.5
llama3.2:1b (1.2B, Q8_0)	10.0 → 63.3	16.7 → 40.0	58.3 → 33.3	36.2 → 33.7	1.9
qwen2.5:1.5b (1.5B, Q4_K_M)	10.0 → 80.0	16.7 → 50.0	58.3 → 41.7	21.4 → 19.9	2.1
smollm2:1.7b (1.7B, Q8_0)	20.0 → 73.3	13.3 → 36.7	33.3 → 41.7	15.2 → 13.8	6.4
granite3.3:2b (2.5B, Q4_K_M)	16.7 → 76.7	26.7 → 43.3	58.3 → 58.3	14.2 → 12.2	8.5
qwen2.5:3b (3.1B, Q4_K_M)	6.7 → 80.0	33.3 → 43.3	25.0 → 33.3	13.3 → 11.0	4.2
falcon3:3b (3.2B, Q4_K_M)	10.0 → 70.0	13.3 → 50.0	33.3 → 66.7	14.2 → 14.5	6.6
llama3.2:3b (3.2B, Q4_K_M)	23.3 → 73.3	16.7 → 56.7	75.0 → 75.0	11.8 → 10.4	4.2
phi4-mini:3.8b (3.8B, Q4_K_M)	13.3 → 80.0	10.0 → 40.0	83.3 → 91.7	12.6 → 10.7	9.4
gemma3:4b (4.3B, Q4_K_M)	20.0 → 73.3	20.0 → 53.3	75.0 → 50.0	9.9 → 9.1	5.6
mistral:7b (7.2B, Q4_K_M)	30.0 → 76.7	13.3 → 43.3	33.3 → 50.0	5.4 → 5.1	10.8
olmo2:7b (7.3B, Q4_K_M)	23.3 → 76.7	36.7 → 58.6	63.6 → 83.3	5.8 → 5.3	8.9
qwen2.5:7b (7.6B, Q4_K_M)	23.3 → 70.0	36.7 → 60.0	83.3 → 83.3	6.1 → 6.0	8.3
llama3.1:8b (8.0B, Q4_K_M)	20.0 → 73.3	36.7 → 56.7	83.3 → 58.3	5.7 → 5.4	9.3
granite3.3:8b (8.2B, Q4_K_M)	16.7 → 80.0	33.3 → 50.0	58.3 → 72.7	5.0 → 4.6	13.1

Model (parameters, quantisation)	PopQA (%)	HotpotQA (%)	GSM8K (%)	Decode tok/s (PopQA)	s / answer with CAMR (PopQA)
gemma2:9b (9.2B, Q4_0)	33.3 → 76.7	33.3 → 50.0	75.0 → 66.7	6.7 → 6.5	16.5
gemma3:12b (12.2B, Q4_K_M)	33.3 → 79.3	30.0 → 63.0	91.7 → 90.0	3.9 → 3.7	16.2
qwen2.5:14b (14.8B, Q4_K_M)	13.3 → 73.3	46.7 → 63.3	66.7 → 91.7	3.2 → 3.0	17.7
Kimi K3 (cloud), no memory	66.7	53.3	100	—	7.7

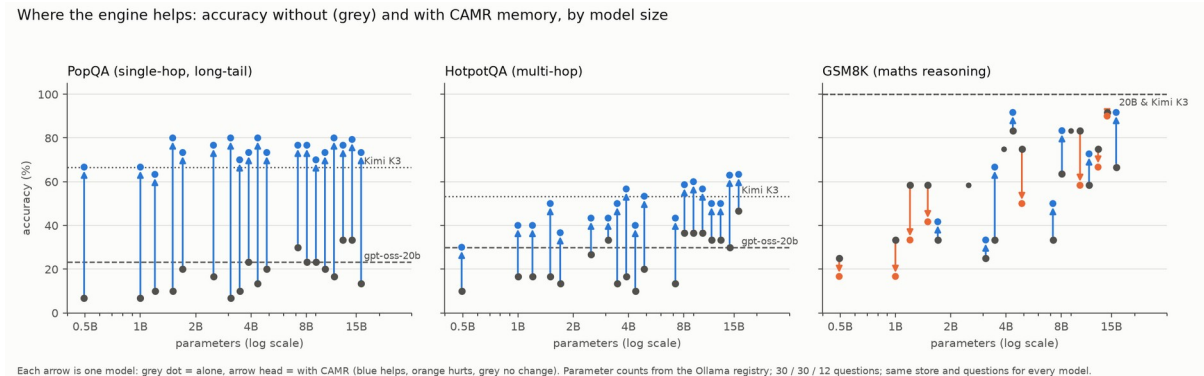


Figure 5.39: Accuracy without and with CAMR by model size, against the Kimi K3 ceiling

All 19 models gained on knowledge tasks: +43 to +73 points on PopQA and +10 to +40 on HotpotQA. With CAMR, 18 of 19 reached or beat Kimi K3 on long-tail facts; only llama3.2:1b (63.3%) fell short. Parametric knowledge barely grew with size: even the 12–15B models alone knew 13–33% of the long-tail facts, while every model from 1.5B upward reached 70–80% with memory. Multi-hop questions depended on the model as a reader. Seven of 19 reached Kimi K3's 53.3%: llama3.2:3b, gemma3:4b and five of the eight 7–15B models, with the best at 63% (qwen2.5:14b, gemma3:12b). Family mattered as much as size, since mistral:7b reached only 43.3%. Speed set the upper bound. On the 4-core CPU, every model of 7B or more with memory answered more slowly than Kimi K3 (8–24 s against 7.7 s on PopQA), so **1.5–4B is the range that suits the engine on this class of device**: faster than the cloud model, at or above it on facts, and at it on multi-hop from about 3B. Worked-example memory for maths helped 8 models, was neutral for 4 and hurt 7, depending on family rather than size, so reasoning routing must be learned per model. A few runs on machine D had one to three queries fail with a runtime error; their accuracy is over the answered questions.

5.8.8 Confirmation at Larger Scale: 200 HotpotQA Questions

To test whether the pilot's multi-hop results held beyond 30 questions, the study was repeated on 200 HotpotQA questions with the read path fixed in advance (similarity, gating and bridging, 512 tokens). Nothing was tuned on these questions. Three small models and Kimi

K3 were run with and without the same notes. 95% confidence intervals come from 2,000 bootstrap resamples of the questions.

Table 5.18: HotpotQA, n = 200: exact match, alone and with CAMR (results/hotpot200)

Model	Alone	With CAMR	Gain (95% CI)	s / answer with CAMR
qwen2.5:0.5b	3.5%	26.5%	+23.0 pts (+17.0 to +29.0)	2.4
qwen2.5:1.5b	10.5%	41.0%	+30.5 pts (+23.0 to +38.0)	2.8
llama3.2:3b	19.5%	54.0%	+34.5 pts (+27.0 to +42.0)	5.8
Kimi K3 (cloud)	51.8%	66.3%	+14.6 pts (+9.0 to +20.6)	9.5

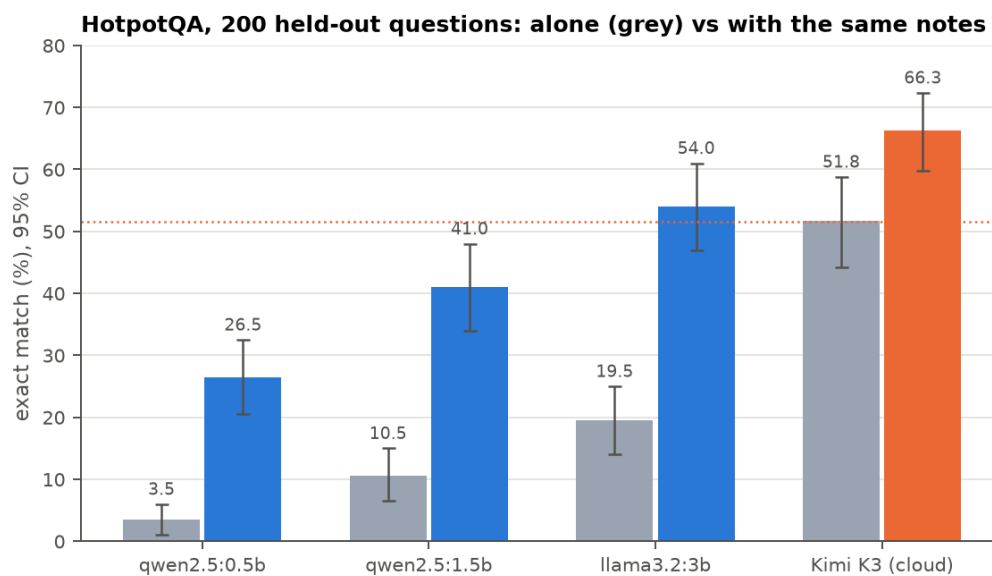


Figure 5.40: HotpotQA, 200 held-out questions: exact match alone and with the same notes, 95% bootstrap intervals

The larger sample confirmed the pilot. Every small model gained substantially, and every 95% interval excludes zero. With CAMR, llama3.2:3b reached 54.0% exact match against Kimi K3's closed-book 51.8% (n = 199), and answered in 5.8 s against 18.4 s. On multi-hop questions, a 3B model on a laptop CPU with memory therefore matches a frontier cloud model at about a third of the answer time. qwen2.5:1.5b reached 41.0% and qwen2.5:0.5b 26.5%. The ordering by size is the pilot's finding that multi-hop gains depend on how well the model reads the evidence. With the same notes, Kimi K3 rose from 51.8% to 66.3%. The residual gap between the best small model and the frontier model reading the same evidence is the share of multi-hop performance that memory cannot buy. Compared with the pilot (30 questions: 30.0%, 50.0% and 56.7% for the three models with memory), the estimates moved

by at most 9 points, within the pilot's uncertainty. Accuracy was identical when the 0.5B run was repeated for clean timings (26.5% both times).

5.8.9 Learning When to Use Memory and When to Escalate

A single fixed configuration cannot be best for every question. The learned policy chose an action per question from pre-answer features and was evaluated on the 72 held-out questions (Table 5.19, Figure 5.41).

Table 5.19: Engine policies on the held-out split (results/learn/tables/rl_policies.md, rl_cascade.md)

Policy	Accuracy	Sent to cloud	Mean s / answer
Always 0.5B alone	13.9%	0%	1.1
Always 0.5B + 128 tokens	41.7%	0%	1.2
Always 0.5B + 512 tokens	45.8%	0%	2.2
Always Kimi K3	63.9%	100%	12.6
Learned router (ridge, cloud cost 0.3)	61.1%	16.7%	2.2
Learned router (LinUCB, after 5 epochs)	56.9%	16.7%	2.2
Router + grounding cascade	65.3%	29.2%	3.1
Oracle (best action in hindsight)	79.2%	23.6%	2.6

The engine learns when to use memory and when to ask the cloud: near-Kimi accuracy, 83% on-device

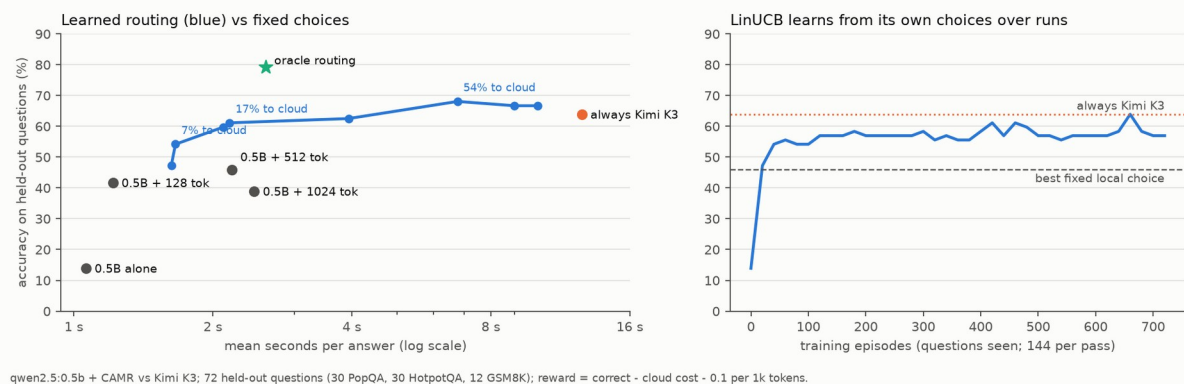


Figure 5.41: Learned routing compared with fixed choices, and the LinUCB learning curve over 5 epochs

The policy learned without supervision to send every reasoning question to the cloud (12 of 12, all correct) and to answer factual questions locally with a small budget. With the grounding cascade, which answers locally first and escalates when the answer's words are not found in its notes, the hybrid reached 65.3% against always-Kimi's 63.9%. 71% of questions never left the device, and mean latency was 4.0 times lower. LinUCB, which sees only the outcome of the action it chose, rose from 13.9% to 54.2% held-out accuracy within the first 40 episodes and then fluctuated between 54% and 64% over the five epochs. The gap to the

oracle (79.2%) is mostly on multi-hop questions, where pre-answer features cannot predict whether the small model will read the evidence correctly.

5.8.10 Cost on the Device (Research Question v)

On machine A, median end-to-end latency was 188 ms for the 0.5B model alone and 375 ms with CAMR. The engine added a median of 187 ms and about 536 prompt tokens. Peak memory was 712 MB for the engine and 1987 MB for the model runtime, and the store file was 47.8 MB. NFR-02 was **not met**: retrieval was 56% of answer time against a 10% target. In isolation the engine's retrieval took 29 ms (embedding 21.5 ms, search 6.2 ms, bridging 1.2 ms, packing 0.4 ms). The rest was contention between the embedder and the model runtime for the same four cores. Partitioning the threads cut the p90 from 157 ms to 50 ms. Because a 0.5B model generates in about 180 ms, the 10% target is reachable only for models whose generation takes 450 ms or more, roughly 1.5B parameters and above on this CPU.

The large local alternative behaved worse on the same device. gpt-oss:20b decoded at about 6.3 tokens/s and needed 19.5–38 s per answer. When a 0.7 GB process ran beside it, it collapsed to 0.4–0.9 tokens/s as its memory-mapped weights were evicted. On a 16 GB device, a small model with external memory is the practical design, not merely the cheaper one.

5.8.11 Responsiveness of the Application

The benchmark profile above measured the engine. A user of CAMR Personal waits for something else: the first word of the answer. Three changes were made to shorten that wait, and each was measured with real models through Ollama on the same 4-core CPU class (scripts/app_latency.py, results in docs/results/app_latency.json). (1)

Warm-up. Ollama loads a model on first use, which dominated the first answer. The app now loads the chosen model in the background when it starts and when the user switches model, while the user is still typing. (2) **Time to first word** is recorded for every answer and shown under it, because streaming makes this, not the total time, the wait a user notices. (3)

Query-embedding cache. A repeated question reuses its embedding instead of calling the embedding model again. The memory held 282 notes: the five demonstration notes and the PopQA documents, with six questions per setting.

Table 5.20: Time a CAMR Personal user waits, per model (budget 384 tokens; medians over six questions)

Model	First word after reboot	First word, cold (cached file)	First word after warm-up	Warm-up (in background)	First word, typical	Whole answer, typical	Memory lookup
qwen2.5:0.5b	2.4 s	1.9 s	0.6 s	1.0 s	0.44 s	0.8 s	218 ms
qwen2.5:1.5b	16.8 s	2.4 s	0.9 s	1.3 s	0.55 s	1.7 s	243 ms
llama3.2:3b	172.7 s	5.0 s	1.7 s	2.4 s	0.85 s	2.6 s	263 ms

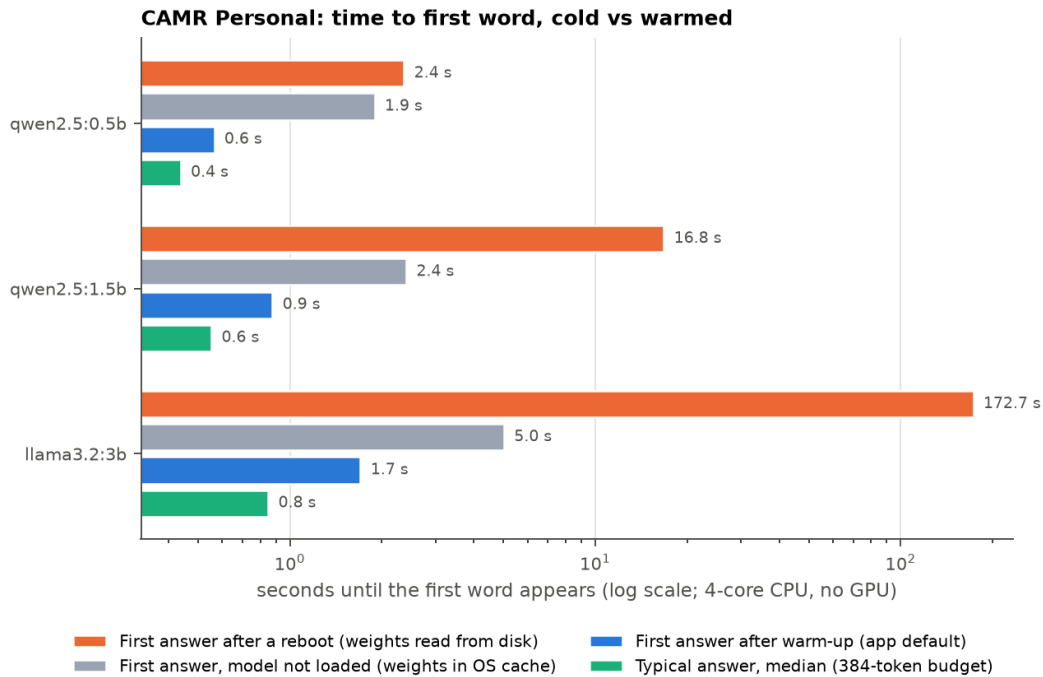


Figure 5.42: Time to the first word: cold start, after the app's warm-up, and a typical answer

Table 5.21: Effect of the memory budget on waiting time (medians)

Model	Budget (tokens)	Prompt tokens	First word	Whole answer
qwen2.5:0.5b	128	168	0.41 s	0.8 s
qwen2.5:0.5b	384	168	0.44 s	0.8 s
qwen2.5:0.5b	768	168	0.42 s	0.8 s
qwen2.5:1.5b	128	168	0.57 s	1.8 s
qwen2.5:1.5b	384	168	0.55 s	1.7 s
qwen2.5:1.5b	768	168	0.51 s	1.6 s
llama3.2:3b	128	160	0.78 s	2.4 s
llama3.2:3b	384	160	0.85 s	2.6 s
llama3.2:3b	768	160	0.82 s	2.4 s

Warm-up. With the model unloaded but its file in the operating system's cache, the first word took 1.9 → 0.6 s, 2.4 → 0.9 s, 5.0 → 1.7 s for qwen2.5:0.5b, qwen2.5:1.5b, llama3.2:3b

once the app had warmed the model, which it does while the user types. After a reboot, when the weights must be read from disk, the cold first answer took 173 s for llama3.2:3b on this container's disk; warm-up moves that load out of the conversation, but cannot hide it if the user asks before it finishes. The first warmed answer is still slower than a typical one because nothing of the prompt has been processed yet. **Budget.** The budget made no measurable difference here: on personal notes, gating admitted only the one or two notes that matched, so every budget sent the same prompt (about 168 tokens) and the budget never bound. It matters when many notes are relevant, as in the benchmarks (section 5.8.5). **Lookup.** Inside a conversation the memory lookup took a median of 218–263 ms, against 48 ms with nothing else running: the embedding model and the chat model share the same four cores, the contention already diagnosed in section 5.8.10. A repeated question skipped the embedding and took 2.8 ms instead of 189 ms. Growing memory 21-fold, from 282 to 5,976 notes, raised the lookup only from 48 to 57 ms (vector search 2 → 11 ms), so memory can keep growing well beyond the model's context window without the assistant becoming noticeably slower. In every case the user sees the first word in under 2 s once the model is loaded.

Table 5.22: Memory lookup time as memory grows (nomic-embed-text through Ollama)

Memory	Notes	Lookup (median)	of which embedding	of which vector search
Demonstration + PopQA	282	48 ms	45 ms	2.1 ms
+ HotpotQA paragraphs	5,976	57 ms	42 ms	11.3 ms
Repeated question (cache)	—	2.8 ms	0 (cached)	—

5.8.12 Alignment with the Proposal (Chapter 1)

Each objective and research question of Chapter 1 was checked against what was built and measured. All five research questions are answered (section 6.2). Where the work departed from the proposal, the departure, its reason and its effect on the findings are recorded in Table 5.23, so that no deviation is hidden in the results.

Table 5.23: Deviations from the approved proposal and their effect

Proposal (Chapter 1)	What was done	Reason	Effect on the findings
Obj. ii: a write policy that <i>distills</i> interactions into compact notes	Distilling (structured) policy implemented and unit-tested; experiments used verbatim notes	Benchmark documents are already compact; distilling would add a model call per note and a confound to the retrieval study	Results describe verbatim memory; distillation is untested at scale
Obj. ii: rank notes by similarity, recency <i>and</i> importance (Eq. 4.1)	Eq. 4.1 implemented and evaluated; the default became similarity + gating + bridging	Eq. 4.1 lowered full-support recall by 1.2–5.4 points (5.8.1)	A measured improvement on the proposal; recency is untested on sessioned data
Obj. iii and scope: one	CPU-only cloud containers of	Several identical machines	Accuracy transfers (greedy,

Proposal (Chapter 1)	What was done	Reason	Effect on the findings
physical consumer laptop as the device	laptop class (4 vCPU, 16 GB, no GPU), fully specified	were needed to run the 19-model sweep in time	deterministic engine); latency is indicative, as limitation 1 expected
Scope: small models of 1–3B parameters	All 1–3B models tested, plus 0.5B and 7–14.8B	To find where the useful size range starts and ends	An extension: confirms 1.5–4B as the range that suits the engine (5.8.7)
Scope: a read-only inspector, not an end-user application	Inspector delivered as scoped; CAMR Personal added after the experiments	To put the findings in users' hands and collect feedback	An extension; it adds no claim to the research results
Delimitation: no port to mobile phones	No mobile results are reported; an unrun Android prototype is described as future work (6.4)	Kept within the delimitation	None
NFR-02: retrieval $\leq 10\%$ of answer time	Measured and reported as not met for 0.5B; diagnosed	A 0.5B model answers in about 180 ms	Met only for models of about 1.5B and above (5.8.10)

5.8.13 Limitations

- i. **Sample size.** Pilot samples were 30/30/12 questions per benchmark. One question is 3.3 points, so differences of one or two questions are within noise. Confidence intervals are reported for gap closed. The multi-hop results were confirmed on 200 questions, and the retrieval study used 500 questions per benchmark; the single-hop and reasoning results rest on the pilot sample.
- ii. **Stochastic cloud ceiling.** Kimi accepts only temperature 1, so its scores are single samples. Small-model floors also varied between repeats (20/30 identical answers), so floors are best read as a range.
- iii. **Oracle-like PopQA corpus.** PopQA documents are the subject's own Wikipedia summary, which favours retrieval. HotpotQA, with eight distractors per question and extra distractor paragraphs, is the more realistic test.
- iv. **Benchmark questions are independent.** Recency could not show a benefit. A sessioned benchmark (e.g. LongMemEval) would be needed to evaluate it fairly.
- v. **Write policy.** Only verbatim notes were evaluated end to end. The structured policy is implemented and tested but was not run at scale.
- vi. **Hardware.** All measurements are from CPU cloud containers of laptop class, not from a physical laptop or phone. Decode speeds differ between machines.
- vii. **No user study yet.** The application was tested for correct and safe behaviour, not for usefulness or usability with real users.
- viii. **Policy learning.** The learned policy was trained on 144 questions with costs set by the researcher. Its advantage over always-Kimi is 1.4 points, which is within sampling noise; the robust result is equal accuracy at 71% on-device use and lower latency.

Conclusions, recommendations and future work, including feedback from real users of CAMR Personal and a phone version of the assistant, are presented in Chapter 6.